



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS



ASIGNATURA:

PROFESOR:

PERÍODO ACADÉMICO: 2026-A

PROYECTO

Sistema de Gestión de Bases de Datos para la Optimización Operativa,
Facturación y Auditoría de Zapaterías (SistemaCalzado_2026)



ESTUDIANTES:

Maria Jose Paredes Viracocha

Juan Daniel Vasquez Mer

INDICE

1. PROPÓSITO DE LA PRÁCTICA	2
2. OBJETIVO GENERAL.....	2
3. OBJETIVOS ESPECÍFICOS.....	2
4. INSTRUCCIONES Y ALCANCE.....	2
5. DESARROLLO	3
5.1 Análisis del Negocio	3
5.2 Modelo Entidad-Relación (MER)	4
5.3 Normalización (1FN, 2FN, 3FN)	5
5.4 Modelo Lógico	9
5.5 Modelo Físico (Scripts de Creación).....	10
5.6 Inserción de Datos (DML)	11
5.7 Consultas SQL (20 Consultas)	15
5.8 Vistas	23
5.9 Funciones	26
5.10 Procedimientos Almacenados.....	28
5.11 Triggers	39
5.12 Transacciones.....	43
5.13 Usuarios, Roles y Privilegios.....	47
5.14 Respaldo y Recuperación.....	50
6. PRESENTACIÓN	52
7. RECURSOS NECESARIOS	52
8. CONCLUSIONES	52
9. RECOMENDACIONES	53
10. BIBLIOGRAFÍA.....	53
11. ANEXOS.....	53

1. PROPÓSITO DE LA PRÁCTICA

El presente proyecto nace de una necesidad operativa real: una empresa ecuatoriana dedicada a la industria del calzado ha sido adquirida recientemente por un grupo empresarial chino. Para afrontar esta nueva etapa comercial, la compañía requiere dejar atrás sus sistemas heredados y migrar hacia una nueva base de datos corporativa. La finalidad principal es gestionar de forma centralizada e integral sus ventas (tanto al por mayor como al por menor), la distribución a nivel nacional, el control riguroso de su inventario, la logística y la administración estructurada de su cartera de clientes y proveedores, garantizando en todo momento la integridad y disponibilidad de la información.

2. OBJETIVO GENERAL

Diseñar e implementar una base de datos relacional robusta en MySQL para la empresa de calzado, que permita administrar de manera eficiente y automatizada los módulos de inventario, ventas, clientes, proveedores y logística, asegurando la consistencia, seguridad y alta disponibilidad de los datos corporativos.

3. OBJETIVOS ESPECÍFICOS

- Identificar con precisión las entidades, atributos y relaciones derivadas del caso de estudio.
- Construir el Modelo Entidad-Relación (MER) que refleje la lógica del negocio.
- Aplicar el proceso de normalización de datos hasta alcanzar la Tercera Forma Normal (3FN).
- Codificar el modelo físico en el motor de base de datos MySQL.
- Desarrollar sentencias SQL avanzadas: consultas, vistas, funciones, procedimientos almacenados, triggers y bloques transaccionales.
- Configurar un esquema de seguridad mediante la implementación de usuarios, roles y privilegios.
- Establecer y documentar los protocolos de respaldo y recuperación de la información.

4. INSTRUCCIONES Y ALCANCE

Para el desarrollo de este sistema, se definieron los siguientes parámetros técnicos obligatorios:

- Despliegue de un esquema con un mínimo de 12 tablas interrelacionadas.
- Población de la base de datos con registros de prueba que cumplan los mínimos establecidos (clientes, productos, ventas, etc.).

- Desarrollo de un mínimo de 20 consultas SQL complejas para inteligencia de negocio.
- Implementación de control de acceso basado en roles.
- Documentación gráfica de todos los procesos ejecutados.

5. DESARROLLO

5.1 Análisis del Negocio

Para que el sistema responda adecuadamente a las operaciones diarias, identificamos los siguientes actores y reglas:

Actores del Sistema:

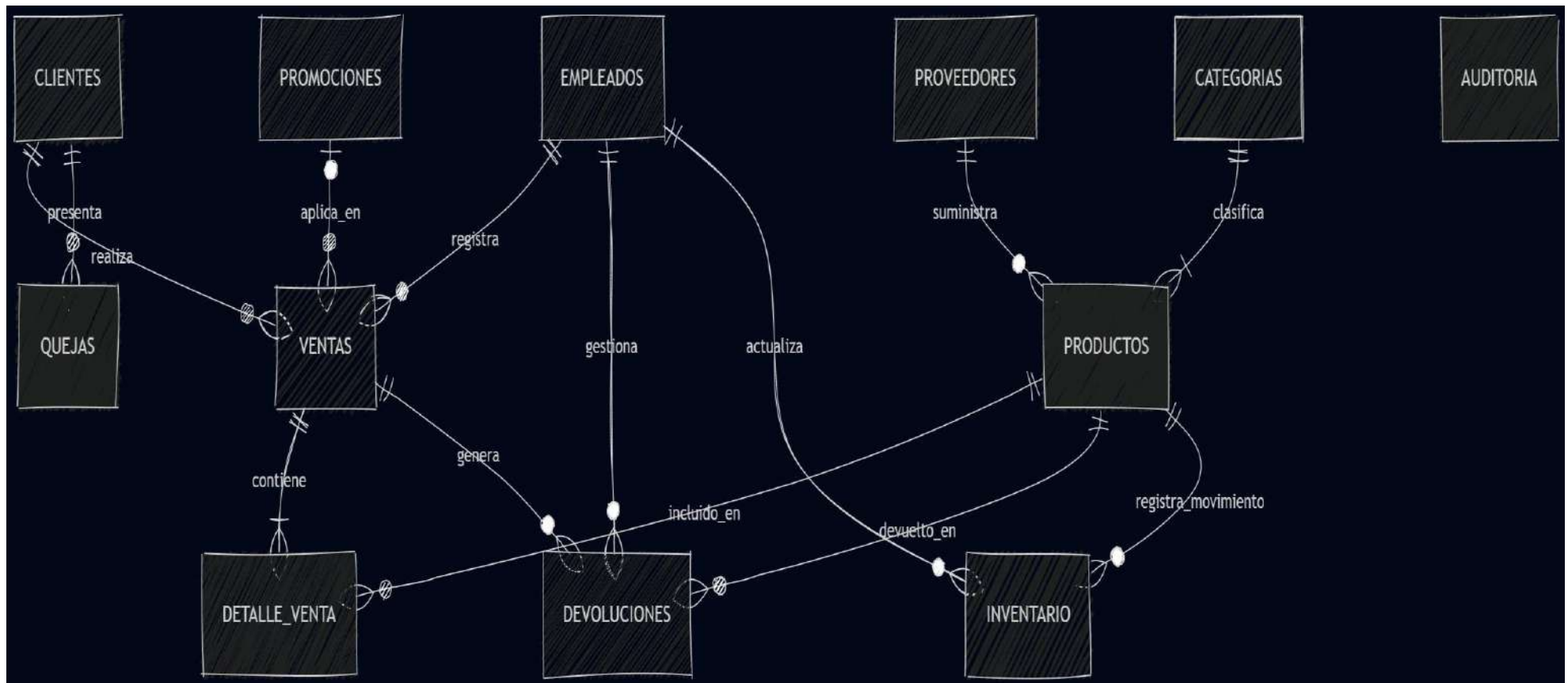
- Administrador (Control total del sistema)
- Gerente (Acceso a reportes y vistas estratégicas)
- Cajero (Facturación y atención directa al cliente)
- Vendedor (Gestión de ventas propias y catálogo)
- Auditor (Revisión de logs y movimientos sin permisos de edición)

Reglas de Negocio Principales:

1. Todo producto del catálogo debe estar vinculado obligatoriamente a una categoría y a un proveedor.
2. Una transacción de venta no puede existir sin al menos un detalle de los productos adquiridos.
3. El sistema debe bloquear cualquier operación que resulte en un stock negativo en el inventario.
4. Toda devolución de mercadería requiere el registro explícito del motivo y el estado del calzado.
5. El personal accederá al sistema únicamente con los privilegios asignados a su rol.
6. Las promociones tienen vigencia definida.

5.2 Modelo Entidad-Relación (MER)

Este modelo representa cómo interactúan los elementos de nuestro negocio de calzado en la vida real, antes de pensar en código. Identificamos entidades principales como **Cientes**, **Empleados**, **Productos** y **Proveedores**. La lógica central del negocio fluye a través de las **Ventas**, donde un cliente adquiere productos atendido por un empleado. Cada venta se desglosa en un **Detalle de Venta** para saber exactamente qué modelos se llevaron. A su vez, el sistema controla el flujo de mercadería mediante un **Inventario** y gestiona los retornos en la entidad de **Devoluciones**. Como un añadido fundamental para el servicio al cliente, se implementó la entidad **Quejas**, la cual tiene una relación de uno a muchos (1:N) con los Clientes, permitiendo registrar todos los reclamos o sugerencias que una persona realice a lo largo del tiempo.



5.3 Normalización (1FN, 2FN, 3FN)

Para el diseño integral de la base de datos, se partió del análisis de los registros históricos heredados de la empresa. A continuación, se presenta un **ejemplo real de las tablas sin modificar (0FN)**, extraído directamente de los archivos planos de Excel que utilizaba la administración anterior antes de la migración al nuevo sistema.

Tabla sin normalizar (0FN):

En este estado original, se evidencia una alta redundancia de datos. La información personal del cliente y las características físicas del producto se repiten innecesariamente por cada artículo vendido dentro de una misma transacción.

idFactura	fecha	cliente_identificacion	cliente_nombres	producto_sku	producto_nombre	producto_talla	precio_unitario	cantidad	total
FAC-001	2026-07-08	1711111111	Cliente Uno	SKU001	Runner X	40.0	50.00	1	110.00
FAC-001	2026-07-08	1711111111	Cliente Uno	SKU002	Classic Low	39.0	60.00	1	110.00
FAC-002	2026-07-09	1722222222	Cliente Dos	SKU001	Runner X	40.0	50.00	2	100.00

Primera Forma Normal (1FN):

Para cumplir con la 1FN, se procedió a eliminar los grupos repetidos (múltiples productos facturados en una sola compra). Se creó una clave primaria compuesta y se separó la cabecera de la factura de sus ítems.

Tabla Factura (Cabecera):

idFactura	fecha	cliente_identificacion	cliente_nombres	total
FAC-001	2026-07-08	1711111111	Cliente Uno	110.00
FAC-002	2026-07-09	1722222222	Cliente Dos	100.00

Tabla DetalleFactura (Ítems):

idFactura	producto_sku	producto_nombre	producto_talla	precio_unitario	cantidad
FAC-001	SKU001	Runner X	40.0	50.00	1
FAC-001	SKU002	Classic Low	39.0	60.00	1
FAC-002	SKU001	Runner X	40.0	50.00	2

Segunda Forma Normal (2FN):

Se eliminaron las dependencias parciales. Los atributos descriptivos del calzado (nombre, talla, color, material) no dependen de la factura de venta, sino únicamente de la clave del producto. Toda esta información se migró a la tabla `productos`.

Tabla DetalleFactura (Normalizada):

idFactura	idProducto	precio_unitario	cantidad	subtotal_linea
FAC-001	SKU001	50.00	1	50.00
FAC-001	SKU002	60.00	1	60.00

Tabla Productos (Aún con dependencias de proveedores):

idProducto	sku	nombre_modelo	talla	nombre_proveedor
1	SKU001	Runner X	40.0	Importadora Asiática
2	SKU002	Classic Low	39.0	Calzado Ecuatoriano S.A.

Tercera Forma Normal (3FN):

Finalmente, se eliminaron las dependencias transitivas. Datos como el nombre del proveedor dependían del RUC del proveedor y no del zapato; de igual forma, los datos del cliente dependían de su cédula. Se crearon tablas maestras independientes para estructurar el catálogo definitivo.

Tabla Productos (Limpia):

idProducto	sku	nombre_modelo	talla	idProveedor
1	SKU001	Runner X	40.0	2
2	SKU002	Classic Low	39.0	1

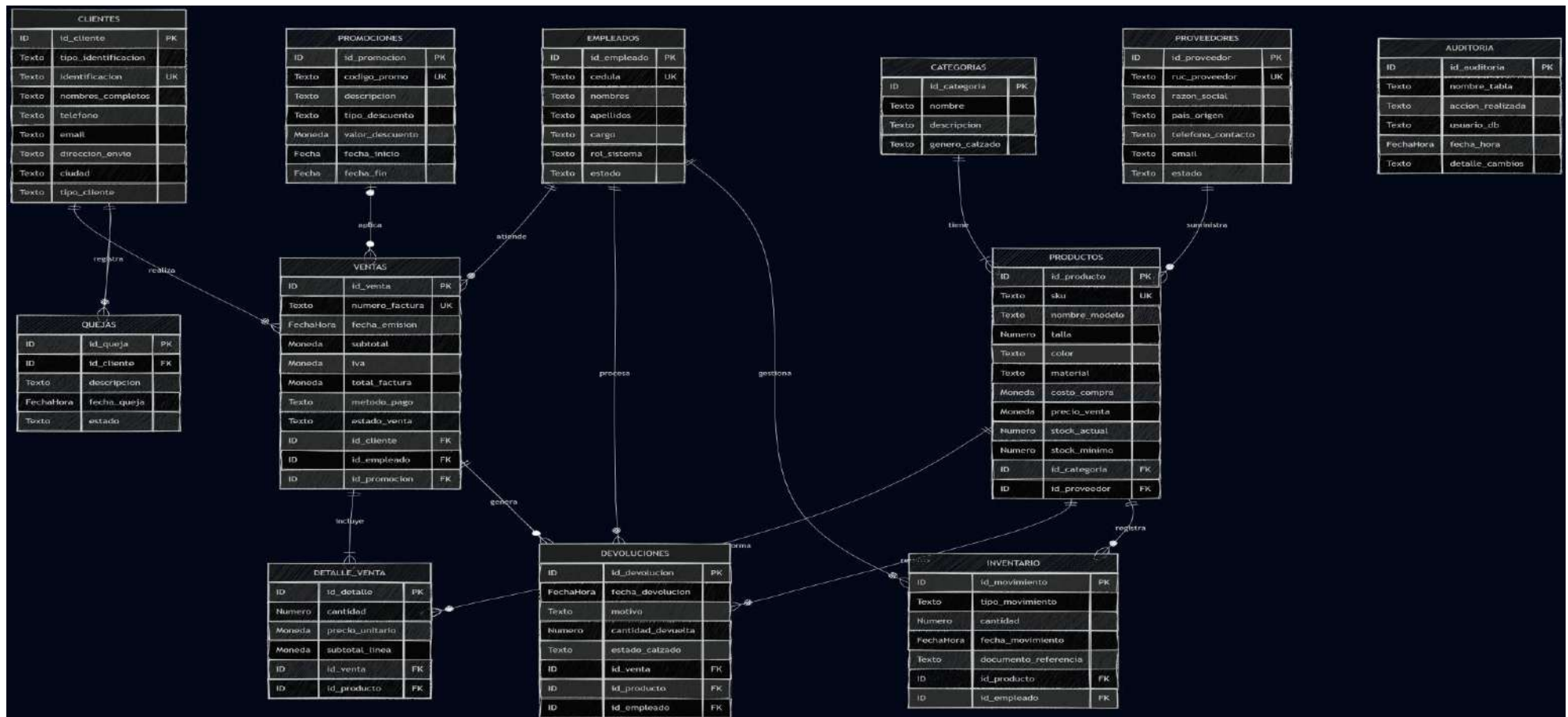
Tabla Proveedores (Nueva Entidad Autónoma):

idProveedor	ruc_proveedor	razon_social
1	1791234567001	Calzado Ecuatoriano S.A.
2	0992345678001	Importadora Asiática

Resultado: El esquema final alcanzó la 3FN, dividiendo la información plana inicial en un sistema relacional de 11 tablas, garantizando la integridad de los datos y optimizando el almacenamiento

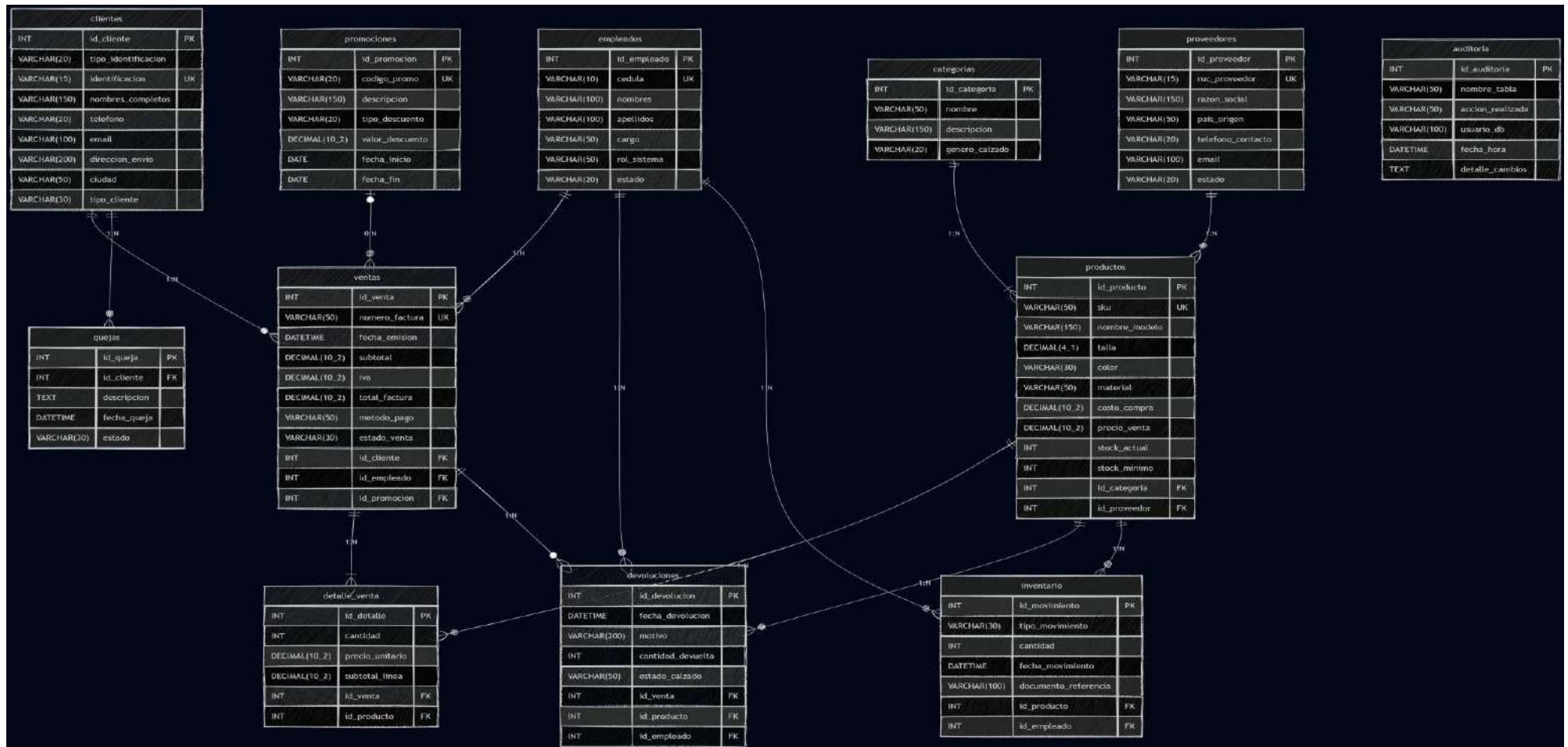
5.4 Modelo Lógico

En esta fase, transformamos las entidades conceptuales en tablas estructuradas con sus respectivos atributos y llaves. Cada tabla tiene una Clave Primaria (PK) que la identifica de forma única, como el `id_producto` o el `id_cliente`. Para conectar la información, utilizamos Claves Foráneas (FK). Por ejemplo, la tabla `ventas` hereda las llaves de `clientes`, `empleados` y `promociones` para saber quién compró, quién vendió y si hubo descuento. Asimismo, estructuramos la nueva tabla `quejas`, asignándole su propia PK (`id_queja`) y conectándola directamente con el cliente a través de la FK `id_cliente`. También definimos qué datos son obligatorios y establecimos la tabla de auditoria como un registro independiente para monitorear los movimientos de los usuarios en el sistema.



5.5 Modelo Físico (Scripts de Creación)

El modelo físico traduce nuestra estructura al código de MySQL definiendo tipos de datos exactos: `VARCHAR` para textos cortos, `DECIMAL` para valores monetarios exactos y `DATETIME` para registrar fechas. Para la nueva tabla de `quejas`, implementamos el tipo de dato `TEXT` para permitir descripciones largas sin límite. Finalmente, se aplicaron las restricciones `FOREIGN KEY` en todo el script para garantizar la integridad de la información y evitar registros huérfanos.



5.6 Inserción de Datos (DML)

Captura de pantalla del Terminal con la ejecución correcta de inserción de datos:

✓	17	22:23:02	INSERT INTO categorias (nombre, descripcion, genero_calzado) VALUES ('Deportivo', 'Calzado para ...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
✓	18	22:23:02	INSERT INTO proveedores (ruc_proveedor, razon_social, pais_origen, telefono_contacto, email) VAL...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
✓	19	22:23:02	INSERT INTO empleados (cedula, nombres, apellidos, cargo, rol_sistema) VALUES ('1712345678', 'J...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
✓	20	22:23:02	INSERT INTO promociones (codigo_promo, descripcion, tipo_descuento, valor_descuento, fecha_inic...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
✓	21	22:23:02	INSERT INTO clientes (tipo_identificacion, identificacion, nombres_completos, telefono, email, direccio...	32 row(s) affected Records: 32 Duplicates: 0 Warnings: 0
✓	22	22:23:02	INSERT INTO productos (sku, nombre_modelo, talla, color, material, costo_compra, precio_venta, sto...	22 row(s) affected Records: 22 Duplicates: 0 Warnings: 0
✓	23	22:23:02	INSERT INTO ventas (numero_factura, subtotal, iva, total_factura, metodo_pago, id_cliente, id_emple...	52 row(s) affected Records: 52 Duplicates: 0 Warnings: 0
✓	24	22:23:02	INSERT INTO detalle_venta (cantidad, precio_unitario, subtotal_linea, id_venta, id_producto) VALUE...	52 row(s) affected Records: 52 Duplicates: 0 Warnings: 0
✓	25	22:23:02	INSERT INTO devoluciones (motivo, cantidad_devuelta, estado_calzado, id_venta, id_producto, id_e...	22 row(s) affected Records: 22 Duplicates: 0 Warnings: 0
✓	26	22:23:02	INSERT INTO inventario (tipo_movimiento, cantidad, documento_referencia, id_producto, id_emplead...	22 row(s) affected Records: 22 Duplicates: 0 Warnings: 0

Verificación de tabla categorias:

287 • `SELECT * FROM categorias;`

Result Grid

Filter Rows:

Edit:   

Export/Import:  

Wrap Cell Content: 

	id_categoria	nombre	descripcion	genero_calzado
▶	1	Deportivo	Calzado para correr	Unisex
	2	Casual	Uso diario	Unisex
	3	Formal	Oficina	Hombre
	4	Escolar	Colegiales	Niños
	5	Plataformas	Moda	Mujer
•	NULL	NULL	NULL	NULL

Verificación de tabla proveedores:

288 • `SELECT * FROM proveedores;`

289 • `SELECT * FROM empleados;`

Result Grid

Filter Rows:

Edit:   

Export/Import:  

Wrap Cell Contents: 

	id_proveedor	ruc_proveedor	razon_social	pais_origen	telefono_contacto	email	estado
▶	1	1791234567001	Calzado Ecuatoriano S.A.	Ecuador	022345678	v@calzadoec.com	Activo
	2	0992345678001	Importadora Asiática	China	042987654	i@asiatica.cn	Activo
	3	1893456789001	Cuero Andino Ltda.	Ecuador	032112233	c@cueroandino.ec	Activo
	4	0194567890001	Suelas del Sur	Ecuador	072345678	s@suelas.com	Activo
	5	1795678901001	Global Shoes	Panamá	022667788	i@globalshoes.pa	Activo
	6	0996789012001	Dist. Guayas	Ecuador	042778899	v@distguayas.ec	Activo
	7	1797890123001	Deportes Cia.	Colombia	022889900	d@ymas.co	Activo
	8	0198901234001	Manu Cuenca	Ecuador	072990011	f@mcuenca.ec	Activo
	9	1799012345001	Zapato Premium	Brasil	022112233	p@zapatos.br	Activo
	10	0990123456001	Industrial Ec	Ecuador	042223344	i@calzado.ec	Activo
	11	1191122334001	Loja Shoes	Ecuador	072111222	l@lojashoes.ec	Activo
	12	1792233445001	Textil Foot	Perú	022334455	t@textilfoot.pe	Activo
	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Verificación de tabla empleados:

289 • `SELECT * FROM empleados;`
 290 • `SELECT * FROM promociones;`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap C

	id_empleado	cedula	nombres	apellidos	cargo	rol_sistema	estado
1	1	1712345678	Juan	Perez	Gerente	usuario_admin	Activo
2	2	1723456789	Maria	Jose	Operaciones	usuario_admin	Activo
3	3	1734567890	Carlos	Mendoza	Vendedor	usuario_vendedor	Activo
4	4	1745678901	Ana	Torres	Cajera	usuario_cajero	Activo
5	5	1756789012	Luis	Paredes	Vendedor	usuario_vendedor	Activo
6	6	1767890123	Sofia	Rios	Auditora	usuario_auditor	Activo
7	7	1778901234	Miguel	Ortega	Gerente	usuario_gerente	Activo
8	8	1789012345	Elena	Castro	Cajera	usuario_cajero	Activo
9	9	1790123456	Andres	Vega	Vendedor	usuario_vendedor	Activo
10	10	1701234567	Lucia	Gomez	Logística	usuario_vendedor	Activo
11	11	1711234567	Pedro	Luna	Vendedor	usuario_vendedor	Activo
12	12	172234455	Marta	Diaz	Cajera	usuario_cajero	Activo
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Verificación de tabla promociones:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: IA

	id_promocion	codigo_promo	descripcion	tipo_descuento	valor_descuento	fecha_inicio	fecha_fin
1	1	VERANO	Verano	Porcentaje	15.00	2026-06-01	2026-08-31
2	2	MADRES	Madres	Porcentaje	20.00	2026-05-01	2026-05-15
3	3	PADRES	Padres	Fijo	5.00	2026-06-01	2026-06-20
4	4	REGRESO	Escolar	Porcentaje	10.00	2026-08-15	2026-09-15
5	5	BLACKFRI	Black Friday	Porcentaje	50.00	2026-11-20	2026-11-30
6	6	NAVIDAD	Navidad	Porcentaje	25.00	2026-12-01	2026-12-25
7	7	LIQUIDA	Liquidación	Fijo	10.00	2026-01-01	2026-01-31
8	8	CUMPLE	Cumpleaños	Fijo	15.00	2026-01-01	2026-12-31
9	9	MAYORISTA	Por mayor	Porcentaje	12.00	2026-01-01	2026-12-31
10	10	FLASH	Flash	Porcentaje	30.00	2026-10-10	2026-10-11
11	11	INVIERNO	Invierno	Porcentaje	15.00	2026-01-01	2026-03-31
12	12	PRIMAVERA	Primavera	Porcentaje	10.00	2026-04-01	2026-05-31
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Verificación de tabla clientes:

292 -- 2. Verificar Clientes y Productos
 293 • `SELECT * FROM clientes;`
 294 • `SELECT * FROM productos;`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Contents: IA

	id_cliente	tipo_identificacion	identificacion	nombres_completos	telefono	email	direccion_envio	ciudad	tipo_cliente
1	1	Cedula	1711111111	Cliente Uno	0991111111	c1@mail.com	Dir 1	Quito	Minorista
2	2	Cedula	1722222222	Cliente Dos	0992222222	c2@mail.com	Dir 2	Guayaquil	Minorista
3	3	Cedula	1733333333	Cliente Tres	0993333333	c3@mail.com	Dir 3	Cuenca	Minorista
4	4	Cedula	1744444444	Cliente Cuatro	0994444444	c4@mail.com	Dir 4	Ambato	Minorista
5	5	Cedula	1755555555	Cliente Cinco	0995555555	c5@mail.com	Dir 5	Quito	Minorista
6	6	Cedula	1766666666	Cliente Seis	0996666666	c6@mail.com	Dir 6	Loja	Minorista
7	7	Cedula	1777777777	Cliente Siete	0997777777	c7@mail.com	Dir 7	Quito	Minorista
8	8	Cedula	1788888888	Cliente Ocho	0998888888	c8@mail.com	Dir 8	Manta	Minorista
9	9	Cedula	1799999999	Cliente Nueve	0999999999	c9@mail.com	Dir 9	Ibarra	Minorista
10	10	Cedula	1700000000	Cliente Diez	0990000000	c10@mail.com	Dir 10	Quito	Minorista
11	11	Cedula	1710101010	Cliente Once	0991010101	c11@mail.com	Dir 11	Guayaquil	Minorista
12	12	Cedula	1712121212	Cliente Doce	0991212121	c12@mail.com	Dir 12	Cuenca	Minorista
13	13	Cedula	1713131313	Cliente Trece	0991313131	c13@mail.com	Dir 13	Ambato	Minorista
14	14	Cedula	1714141414	Cliente Catorce	0991414141	c14@mail.com	Dir 14	Quito	Minorista
15	15	Cedula	1715151515	Cliente Quince	0991515151	c15@mail.com	Dir 15	Loja	Minorista

Verificacion de la tabla productos:

294 • `SELECT * FROM productos;`
295

id_producto	sku	nombre_modelo	talla	color	material	costo_compra	precio_venta	stock_actual	stock_minimo	id_categoria	id_proveedor
1	SKU001	Runner X	40.0	Rojo	Sintetico	25.00	50.00	100	10	1	2
2	SKU002	Classic Low	39.0	Blanco	Cuero	30.00	60.00	80	10	2	1
3	SKU003	Oxford Pro	42.0	Negro	Cuero	40.00	85.00	50	5	3	3
4	SKU004	Colegial Basic	35.0	Negro	Cuero	15.00	35.00	150	20	4	1
5	SKU005	Heels Elegance	37.0	Nude	Sintetico	20.00	45.00	60	10	5	5
6	SKU006	Runner Pro	41.0	Azul	Malla	28.00	55.00	90	10	1	2
7	SKU007	Sneaker High	38.0	Negro	Lona	18.00	40.00	110	15	2	4
8	SKU008	Derby Classic	43.0	Cafe	Cuero	42.00	90.00	45	5	3	3
9	SKU009	Kids Play	32.0	Blanco	Sintetico	12.00	28.00	200	25	4	1
10	SKU010	Platform Star	36.0	Plata	Sintetico	22.00	48.00	70	10	5	5
11	SKU011	Trail Max	44.0	Verde	Imperm...	35.00	75.00	40	5	1	6
12	SKU012	Loafer Comfort	40.5	Marron	Gamuza	32.00	65.00	55	10	2	8
13	SKU013	Monk Strap	41.5	Negro	Cuero	45.00	95.00	30	5	3	3
14	SKU014	School Girl	34.0	Azul ...	Cuero	16.00	36.00	130	20	4	1

Verificacion de tabla ventas:

297 • `SELECT * FROM ventas;`
298 • `SELECT * FROM detalle_venta;`

id_venta	numero_factura	fecha_emision	subtotal	iva	total_factura	metodo_pago	estado_venta	id_cliente	id_empleado	id_promocion
1	FAC-001	2026-07-08 22:23:02	50.00	7.50	57.50	Efectivo	Completada	1	3	NUL
2	FAC-002	2026-07-08 22:23:02	60.00	9.00	69.00	Tarjeta	Completada	2	4	1
3	FAC-003	2026-07-08 22:23:02	85.00	12.75	97.75	Efectivo	Completada	3	5	NUL
4	FAC-004	2026-07-08 22:23:02	35.00	5.25	40.25	Transferencia	Completada	4	3	2
5	FAC-005	2026-07-08 22:23:02	45.00	6.75	51.75	Efectivo	Completada	5	4	NUL
6	FAC-006	2026-07-08 22:23:02	55.00	8.25	63.25	Tarjeta	Completada	6	5	NUL
7	FAC-007	2026-07-08 22:23:02	40.00	6.00	46.00	Efectivo	Completada	7	3	NUL
8	FAC-008	2026-07-08 22:23:02	90.00	13.50	103.50	Transferencia	Completada	8	4	3
9	FAC-009	2026-07-08 22:23:02	28.00	4.20	32.20	Efectivo	Completada	9	5	NUL
10	FAC-010	2026-07-08 22:23:02	48.00	7.20	55.20	Tarjeta	Completada	10	3	NUL
11	FAC-011	2026-07-08 22:23:02	75.00	11.25	86.25	Efectivo	Completada	11	4	NUL
12	FAC-012	2026-07-08 22:23:02	65.00	9.75	74.75	Transferencia	Completada	12	5	4
13	FAC-013	2026-07-08 22:23:02	95.00	14.25	109.25	Efectivo	Completada	13	3	NUL
14	FAC-014	2026-07-08 22:23:02	36.00	5.40	41.40	Tarjeta	Completada	14	4	NUL

Verificacion de tabla detalle_ventas:

298 • `SELECT * FROM detalle_venta;`
299 • `SELECT * FROM devoluciones;`

id_detalle	cantidad	precio_unitario	subtotal_linea	id_venta	id_producto
1	1	50.00	50.00	1	1
2	1	60.00	60.00	2	2
3	1	85.00	85.00	3	3
4	1	35.00	35.00	4	4
5	1	45.00	45.00	5	5
6	1	55.00	55.00	6	6
7	1	40.00	40.00	7	7
8	1	90.00	90.00	8	8
9	1	28.00	28.00	9	9
10	1	48.00	48.00	10	10
11	1	75.00	75.00	11	11
12	1	65.00	65.00	12	12
13	1	95.00	95.00	13	13
14	1	36.00	36.00	14	14

Verificacion de tabla devoluciones:

299 • `SELECT * FROM devoluciones;`
 300 • `SELECT * FROM inventario;`

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content: [IA](#)

	id_devolucion	fecha_devolucion	motivo	cantidad_devuelta	estado_calzado	id_venta	id_producto	id_empleado
▶	1	2026-07-08 22:23:02	Talla incorrecta	1	Nuevo	1	1	3
	2	2026-07-08 22:23:02	Defecto de fabrica	1	Defectuoso	2	2	4
	3	2026-07-08 22:23:02	No le gusto	1	Nuevo	3	3	5
	4	2026-07-08 22:23:02	Talla incorrecta	1	Nuevo	4	4	3
	5	2026-07-08 22:23:02	Mancha en el cuero	1	Defectuoso	5	5	4
	6	2026-07-08 22:23:02	Suela despegada	1	Defectuoso	6	6	5
	7	2026-07-08 22:23:02	Cambio por otro color	1	Nuevo	7	7	3
	8	2026-07-08 22:23:02	Talla incorrecta	1	Nuevo	8	8	4
	9	2026-07-08 22:23:02	Defecto costura	1	Defectuoso	9	9	5
	10	2026-07-08 22:23:02	No le gusto	1	Nuevo	10	10	3
	11	2026-07-08 22:23:02	Talla incorrecta	1	Nuevo	11	11	4
	12	2026-07-08 22:23:02	Defecto de fabrica	1	Defectuoso	12	12	5
	13	2026-07-08 22:23:02	Cambio modelo	1	Nuevo	13	13	3
	14	2026-07-08 22:23:02	Talla incorrecta	1	Nuevo	14	14	4
	15	2026-07-08 22:23:02	Mancha	1	Defectuoso	15	15	5

Verificacion de tabla inventario:

299 • `SELECT * FROM devoluciones;`
 300 • `SELECT * FROM inventario;`

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content: [IA](#)

	id_devolucion	fecha_devolucion	motivo	cantidad_devuelta	estado_calzado	id_venta	id_producto	id_empleado
▶	1	2026-07-08 22:23:02	Talla incorrecta	1	Nuevo	1	1	3
	2	2026-07-08 22:23:02	Defecto de fabrica	1	Defectuoso	2	2	4
	3	2026-07-08 22:23:02	No le gusto	1	Nuevo	3	3	5
	4	2026-07-08 22:23:02	Talla incorrecta	1	Nuevo	4	4	3
	5	2026-07-08 22:23:02	Mancha en el cuero	1	Defectuoso	5	5	4
	6	2026-07-08 22:23:02	Suela despegada	1	Defectuoso	6	6	5
	7	2026-07-08 22:23:02	Cambio por otro color	1	Nuevo	7	7	3
	8	2026-07-08 22:23:02	Talla incorrecta	1	Nuevo	8	8	4
	9	2026-07-08 22:23:02	Defecto costura	1	Defectuoso	9	9	5
	10	2026-07-08 22:23:02	No le gusto	1	Nuevo	10	10	3
	11	2026-07-08 22:23:02	Talla incorrecta	1	Nuevo	11	11	4
	12	2026-07-08 22:23:02	Defecto de fabrica	1	Defectuoso	12	12	5
	13	2026-07-08 22:23:02	Cambio modelo	1	Nuevo	13	13	3
	14	2026-07-08 22:23:02	Talla incorrecta	1	Nuevo	14	14	4
	15	2026-07-08 22:23:02	Mancha	1	Defectuoso	15	15	5

5.7 Consultas SQL (20 Consultas)

Consulta 1: Listado de clientes

<pre> SELECT id_cliente AS IdCliente, nombres_completos AS Nombres, telefono AS Telefono, email AS Correo, ciudad AS Provincia FROM clientes; </pre>	Result Grid Filter Rows: Export: Wrap Cell Content:				
	IdCliente	Nombres	Telefono	Correo	Provincia
	1	Cliente Uno	0991111111	c1@mail.com	Quito
	2	Cliente Dos	0992222222	c2@mail.com	Guayaquil
	3	Cliente Tres	0993333333	c3@mail.com	Cuenca
	4	Cliente Cuatro	0994444444	c4@mail.com	Ambato
	5	Cliente Cinco	0995555555	c5@mail.com	Quito
	6	Cliente Seis	0996666666	c6@mail.com	Loja
	7	Cliente Siete	0997777777	c7@mail.com	Quito
	8	Cliente Ocho	0998888888	c8@mail.com	Manta
	9	Cliente Nueve	0999999999	c9@mail.com	Ibarra
	10	Cliente Diez	0990000000	c10@mail.com	Quito
	11	Cliente Once	0991010101	c11@mail.com	Guayaquil
	12	Cliente Doce	0991212121	c12@mail.com	Cuenca
	13	Cliente Trece	0991313131	c13@mail.com	Ambato
	14	Cliente Catorce	0991414141	c14@mail.com	Quito
	15	Cliente Quince	0991515151	c15@mail.com	Loja
	16	Cliente Dieciseis	0991616161	c16@mail.com	Quito
	17	Cliente Diecisi...	0991717171	c17@mail.com	Manta
	18	Cliente Diecio...	0991818181	c18@mail.com	Ibarra
	19	Cliente Diecin...	0991919191	c19@mail.com	Quito
	20	Cliente Veinte	0992020202	c20@mail.com	Guayaquil
	21	Cliente Vintiuno	0992121212	c21@mail.com	Cuenca

Consulta 2: Productos disponibles

<pre> SELECT p.id_producto AS IdProducto, p.nombre_modelo AS NombreProducto, c.nombre AS TipoProducto, p.precio_venta AS Precio, p.stock_actual AS StockActual FROM productos p JOIN categorias c ON p.id_categoria = c.id_categoria WHERE p.stock_actual > 0; </pre>	Result Grid Filter Rows: Export: Wrap Cell Content:				
	IdProducto	NombreProducto	TipoProducto	Precio	StockActual
	1	Runner X	Deportivo	50.00	100
	6	Runner Pro	Deportivo	55.00	90
	11	Trail Max	Deportivo	75.00	40
	16	Marathon Lite	Deportivo	52.00	105
	21	Soccer Cleat	Deportivo	65.00	75
	2	Classic Low	Casual	60.00	80
	7	Sneaker High	Casual	40.00	110
	12	Loafer Comfort	Casual	65.00	55
	17	Slip-On Easy	Casual	30.00	140
	22	Moccasin Relax	Casual	58.00	80
	3	Oxford Pro	Formal	85.00	50
	8	Derby Classic	Formal	90.00	45
	13	Monk Strap	Formal	95.00	30
	18	Chelsea Boot	Formal	80.00	65
	4	Colegial Basic	Escolar	35.00	150
	9	Kids Play	Escolar	28.00	200
	14	School Girl	Escolar	36.00	130
	19	Sport Junior	Escolar	32.00	180
	5	Heels Elegance	Plataformas	45.00	60
	10	Platform Star	Plataformas	48.00	70
	15	Merino Summer	Plataformas	47.00	85

Consulta 3: Ventas por fecha

```
SELECT
    v.id_venta AS IdVenta,
    v.fecha_emision AS
FechaVenta,
    c.nombres_completos AS
Cliente,
    v.total_factura AS
TotalVenta
FROM ventas v
JOIN clientes c ON
v.id_cliente = c.id_cliente
ORDER BY v.fecha_emision
DESC;
```

Result Grid	Filter Rows:	Exports	Wrap Cell Content:
IdVenta	FechaVenta	Cliente	TotalVenta
1	2026-07-08 22:23:02	Cliente Uno	57.50
33	2026-07-08 22:23:02	Cliente Uno	86.25
2	2026-07-08 22:23:02	Cliente Dos	69.00
34	2026-07-08 22:23:02	Cliente Dos	74.75
3	2026-07-08 22:23:02	Cliente Tres	97.75
35	2026-07-08 22:23:02	Cliente Tres	109.25
4	2026-07-08 22:23:02	Cliente Cuatro	40.25
36	2026-07-08 22:23:02	Cliente Cuatro	41.40
5	2026-07-08 22:23:02	Cliente Cinco	51.75
37	2026-07-08 22:23:02	Cliente Cinco	48.30
6	2026-07-08 22:23:02	Cliente Seis	63.25
38	2026-07-08 22:23:02	Cliente Seis	59.80
7	2026-07-08 22:23:02	Cliente Siete	46.00
39	2026-07-08 22:23:02	Cliente Siete	34.50
8	2026-07-08 22:23:02	Cliente Ocho	103.50
40	2026-07-08 22:23:02	Cliente Ocho	92.00
9	2026-07-08 22:23:02	Cliente Nueve	32.20
41	2026-07-08 22:23:02	Cliente Nueve	36.80
10	2026-07-08 22:23:02	Cliente Diez	55.20
42	2026-07-08 22:23:02	Cliente Diez	63.25
11	2026-07-08 22:23:02	Cliente Once	86.25

Consulta 4: Proveedores registrados

```
SELECT
    id_proveedor AS
IdProveedor,
    razon_social AS
NombreProveedor,
    pais_origen AS Pais,
    estado AS TipoProveedor,
    telefono_contacto AS
Telefono
FROM proveedores;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	IdProveedor	NombreProveedor	Pais	TipoProveedor	Telefono
▶	1	Calzado Ecuatoriano S.A.	Ecuador	Activo	022345678
	2	Importadora Asiática	China	Activo	042987654
	3	Cuero Andino Ltda.	Ecuador	Activo	032112233
	4	Suelas del Sur	Ecuador	Activo	072345678
	5	Global Shoes	Panamá	Activo	022667788
	6	Dist. Guayas	Ecuador	Activo	042778899
	7	Deportes Cla.	Colombia	Activo	022889900
	8	Manu Cuenca	Ecuador	Activo	072990011
	9	Zapato Premium	Brasil	Activo	022112233
	10	Industrial Ec	Ecuador	Activo	042223344
	11	Loja Shoes	Ecuador	Activo	072111222
	12	Textil Foot	Perú	Activo	022334455

Consulta 5: Empleados por rol

```
SELECT
    id_empleado AS IdEmpleado,
    CONCAT(nombres, ' ',
apellidos) AS NombreEmpleado,
    cargo AS Cargo,
    rol_sistema AS Rol
FROM empleados
ORDER BY rol_sistema;
```

Result Grid	Filter Rows:	Exports	Wrap Cell Content:
IdEmpleado	NombreEmpleado	Cargo	Rol
1	Juan Perez	Gerente	usuario_admin
2	Maria Jose	Operaciones	usuario_admin
6	Sofia Rios	Auditora	usuario_auditor
4	Ana Torres	Cajera	usuario_cajero
8	Elena Castro	Cajera	usuario_cajero
12	Marta Diaz	Cajera	usuario_cajero
7	Miguel Ortega	Gerente	usuario_gerente
3	Carlos Mendoza	Vendedor	usuario_vendedor
5	Luis Paredes	Vendedor	usuario_vendedor
9	Andres Vega	Vendedor	usuario_vendedor
10	Lucia Gomez	Logistica	usuario_vendedor
11	Pedro Luna	Vendedor	usuario_vendedor

Consulta 6: Clientes con sus compras

```
SELECT
    c.nombres_completos AS
    Cliente,
    v.fecha_emision AS
    FechaVenta,
    v.total_factura AS
    TotalVenta,
    v.metodo_pago AS TipoVenta
FROM clientes c
JOIN ventas v ON c.id_cliente =
v.id_cliente
ORDER BY v.fecha_emision DESC;
```

Cliente	FechaVenta	TotalVenta	TipoVenta
Cliente Uno	2026-07-08 22:23:02	57.50	Efectivo
Cliente Uno	2026-07-08 22:23:02	86.25	Efectivo
Cliente Dos	2026-07-08 22:23:02	69.00	Tarjeta
Cliente Dos	2026-07-08 22:23:02	74.75	Tarjeta
Cliente Tres	2026-07-08 22:23:02	97.75	Efectivo
Cliente Tres	2026-07-08 22:23:02	109.25	Efectivo
Cliente Cuatro	2026-07-08 22:23:02	40.25	Transferencia
Cliente Cuatro	2026-07-08 22:23:02	41.40	Transferencia
Cliente Cinco	2026-07-08 22:23:02	51.75	Efectivo
Cliente Cinco	2026-07-08 22:23:02	48.30	Efectivo
Cliente Seis	2026-07-08 22:23:02	63.25	Tarjeta
Cliente Seis	2026-07-08 22:23:02	59.80	Tarjeta
Cliente Siete	2026-07-08 22:23:02	46.00	Efectivo
Cliente Siete	2026-07-08 22:23:02	34.50	Efectivo
Cliente Ocho	2026-07-08 22:23:02	103.50	Transferencia
Cliente Ocho	2026-07-08 22:23:02	92.00	Transferencia
Cliente Nueve	2026-07-08 22:23:02	32.20	Efectivo
Cliente Nueve	2026-07-08 22:23:02	36.80	Efectivo
Cliente Diez	2026-07-08 22:23:02	55.20	Tarjeta
Cliente Diez	2026-07-08 22:23:02	63.25	Tarjeta

Consulta 7: Ventas con vendedor

```
SELECT
    v.id_venta AS IdVenta,
    c.nombres_completos AS
    Cliente,
    e.nombres AS Vendedor,
    v.fecha_emision AS
    FechaVenta,
    v.total_factura AS TotalVenta
FROM ventas v
JOIN clientes c ON v.id_cliente =
c.id_cliente
JOIN empleados e ON v.id_empleado
= e.id_empleado
ORDER BY v.fecha_emision DESC;
```

IdVenta	Cliente	Vendedor	FechaVenta	TotalVenta
1	Cliente Uno	Carlos	2026-07-08 22:23:02	57.50
4	Cliente Cuatro	Carlos	2026-07-08 22:23:02	40.25
7	Cliente Siete	Carlos	2026-07-08 22:23:02	46.00
10	Cliente Diez	Carlos	2026-07-08 22:23:02	55.20
13	Cliente Trece	Carlos	2026-07-08 22:23:02	109.25
16	Cliente Dieciseis	Carlos	2026-07-08 22:23:02	59.80
19	Cliente Diecinueve	Carlos	2026-07-08 22:23:02	36.80
22	Cliente Vintidos	Carlos	2026-07-08 22:23:02	66.70
25	Cliente Vinticinco	Carlos	2026-07-08 22:23:02	97.75
28	Cliente Vintiocho	Carlos	2026-07-08 22:23:02	63.25
31	Cliente Treintayuno	Carlos	2026-07-08 22:23:02	32.20
34	Cliente Dos	Carlos	2026-07-08 22:23:02	74.75
37	Cliente Cinco	Carlos	2026-07-08 22:23:02	48.30
40	Cliente Ocho	Carlos	2026-07-08 22:23:02	92.00
43	Cliente Once	Carlos	2026-07-08 22:23:02	74.75
46	Cliente Catorce	Carlos	2026-07-08 22:23:02	69.00
49	Cliente Diecisiete	Carlos	2026-07-08 22:23:02	51.75
52	Cliente Veinte	Carlos	2026-07-08 22:23:02	103.50

Consulta 8: Detalle de productos vendidos

```
SELECT
    dv.id_venta AS IdVenta,
    p.nombre_modelo AS
    Producto,
    dv.cantidad AS Cantidad,
    dv.precio_unitario AS
    PrecioUnitario,
    dv.subtotal_linea AS
    Subtotal
FROM detalle_venta dv
JOIN productos p ON
dv.id_producto = p.id_producto
ORDER BY dv.id_venta;
```

IdVenta	Producto	Cantidad	PrecioUnitario	Subtotal
1	Runner X	1	50.00	50.00
2	Classic Low	1	60.00	60.00
3	Oxford Pro	1	85.00	85.00
4	Colegial Basic	1	35.00	35.00
5	Heels Elegance	1	45.00	45.00
6	Runner Pro	1	55.00	55.00
7	Sneaker High	1	40.00	40.00
8	Derby Classic	1	90.00	90.00
9	Kids Play	1	28.00	28.00
10	Platform Star	1	48.00	48.00
11	Trail Max	1	75.00	75.00
12	Loafer Comfort	1	65.00	65.00
13	Monk Strap	1	95.00	95.00
14	School Girl	1	36.00	36.00
15	Wedge Summer	1	42.00	42.00
16	Marathon Lite	1	52.00	52.00
17	Slip-On Easy	1	30.00	30.00
18	Chelsea Boot	1	80.00	80.00

Consulta 9: Productos con proveedor

SELECT
p.nombre_modelo AS Producto,
cat.nombre AS TipoProducto,
pr.razon_social AS Proveedor,
pr.pais_origen AS PaisProveedor
FROM productos p
JOIN categorias cat ON
p.id_categoria = cat.id_categoria
JOIN proveedores pr ON
p.id_proveedor = pr.id_proveedor;

Producto	TipoProducto	Proveedor	PaisProveedor
▶ Runner X	Deportivo	Importadora Asiática	China
Runner Pro	Deportivo	Importadora Asiática	China
Trail Max	Deportivo	Dist. Guayas	Ecuador
Marathon Lite	Deportivo	Importadora Asiática	China
Soccer Cleat	Deportivo	Deportes Cia.	Colombia
Classic Low	Casual	Calzado Ecuatoriano S.A.	Ecuador
Sneaker High	Casual	Suelas del Sur	Ecuador
Loafer Comfort	Casual	Manu Cuenca	Ecuador
Slip-On Easy	Casual	Suelas del Sur	Ecuador
Moccasin Relax	Casual	Manu Cuenca	Ecuador
Oxford Pro	Formal	Cuero Andino Ltda.	Ecuador
Derby Classic	Formal	Cuero Andino Ltda.	Ecuador
Monk Strap	Formal	Cuero Andino Ltda.	Ecuador
Chelsea Boot	Formal	Manu Cuenca	Ecuador
Colegial Basic	Escolar	Calzado Ecuatoriano S.A.	Ecuador
Kids Play	Escolar	Calzado Ecuatoriano S.A.	Ecuador
School Girl	Escolar	Calzado Ecuatoriano S.A.	Ecuador
Sport Junior	Escolar	Calzado Ecuatoriano S.A.	Ecuador

Consulta 10: Devoluciones con cliente y producto

SELECT	Cliente	Producto	FechaDevolucion	Motivo	Cantidad
c.nombres_completos AS	▶ Cliente Uno	Runner X	2026-07-08 22:23:02	Talla incorrecta	1
Cliente,	Cliente Dos	Classic Low	2026-07-08 22:23:02	Defecto de fabrica	1
p.nombre_modelo AS Producto,	Cliente Tres	Oxford Pro	2026-07-08 22:23:02	No le gusto	1
d.fecha_devolucion AS	Cliente Cuatro	Colegial Basic	2026-07-08 22:23:02	Talla incorrecta	1
FechaDevolucion,	Cliente Cinco	Heels Elegance	2026-07-08 22:23:02	Mancha en el cuero	1
d.motivo AS Motivo,	Cliente Seis	Runner Pro	2026-07-08 22:23:02	Suela despegada	1
d.cantidad_devuelta AS	Cliente Siete	Sneaker High	2026-07-08 22:23:02	Cambio por otro color	1
Cantidad	Cliente Ocho	Derby Classic	2026-07-08 22:23:02	Talla incorrecta	1
FROM devoluciones d	Cliente Nueve	Kids Play	2026-07-08 22:23:02	Defecto costura	1
JOIN ventas v ON d.id_venta =	Cliente Diez	Platform Star	2026-07-08 22:23:02	No le gusto	1
v.id_venta	Cliente Once	Trail Max	2026-07-08 22:23:02	Talla incorrecta	1
JOIN clientes c ON v.id_cliente =	Cliente Doce	Loafer Comfort	2026-07-08 22:23:02	Defecto de fabrica	1
c.id_cliente	Cliente Trece	Monk Strap	2026-07-08 22:23:02	Cambio modelo	1
JOIN productos p ON d.id_producto	Cliente Catorce	School Girl	2026-07-08 22:23:02	Talla incorrecta	1
= p.id_producto	Cliente Quince	Wedge Summer	2026-07-08 22:23:02	Mancha	1
ORDER BY d.fecha_devolucion DESC;	Cliente Dieciseis	Marathon Lite	2026-07-08 22:23:02	No le gusto	1
	Cliente Diecisi...	Slip-On Easy	2026-07-08 22:23:02	Cambio por talla	1
	Cliente Diecio...	Chelsea Boot	2026-07-08 22:23:02	Cordon roto	1

Consulta 11. Total vendido por vendedor

```

SELECT e.nombres AS Vendedor,
COUNT(v.id_venta) AS
CantidadVentas,
SUM(v.total_factura) AS
TotalVendido
FROM ventas v
JOIN empleados e ON
v.id_empleado = e.id_empleado
GROUP BY e.id_empleado,
e.nombres
ORDER BY TotalVendido DESC;

```

Result Grid



Filter Rows:

	Vendedor	CantidadVentas	TotalVendido
▶	Carlos	18	1178.75
	Ana	17	1082.15
	Luis	17	1076.40

Consulta 12. Productos más vendidos

```
SELECT p.nombre_modelo AS Producto,
SUM(dv.cantidad) AS
TotalUnidadesVendidas
FROM detalle_venta dv
JOIN productos p ON dv.id_producto
= p.id_producto
GROUP BY p.id_producto,
p.nombre_modelo
ORDER BY TotalUnidadesVendidas
DESC;
```

Producto	TotalUnidadesVendidas
Runner X	3
Classic Low	3
Oxford Pro	3
Colegial Basic	3
Heels Elegance	3
Runner Pro	3
Sneaker High	3
Derby Classic	3
Kids Play	2
Platform Star	2
Trail Max	2
Loafer Comfort	2

Consulta 13. Ventas totales por mes

Un solo mes

```
SELECT
EXTRACT(MONTH FROM
fecha_emision) AS Mes,
COUNT(*) AS TotalVentas,
SUM(total_factura) AS
MontoTotal
FROM ventas
GROUP BY EXTRACT(MONTH FROM
fecha_emision)
ORDER BY Mes;
```

Mes	TotalVentas	MontoTotal
7	52	3337.30

Todos los meses del año (Ejemplo práctico de consulta anual distinta al solicitado en el documento)

```
-- Script adicional para pruebas de ventas anuales
-- 1. Ventas adicionales distribuidas en todo el año 2026
INSERT INTO ventas (numero_factura, fecha_emision, subtotal, iva, total_factura, metodo_pago, id_cliente, id_empleado, id_promocion) VALUES
('FAC-052', '2026-01-15 10:30:00', 120.00, 14.40, 134.40, 'Efectivo', 1, 2, 7),
('FAC-054', '2026-02-20 14:15:00', 80.00, 9.60, 89.60, 'Tarjeta', 2, 4, NULL),
('FAC-055', '2026-03-10 09:45:00', 150.00, 18.00, 168.00, 'Transferencia', 3, 5, 11),
('FAC-056', '2026-04-05 16:20:00', 60.00, 7.20, 67.20, 'Efectivo', 4, 3, 12),
('FAC-057', '2026-05-12 11:10:00', 200.00, 24.00, 224.00, 'Tarjeta', 5, 4, 2),
('FAC-058', '2026-06-18 13:30:00', 90.00, 10.80, 100.80, 'Transferencia', 6, 5, 1),
('FAC-059', '2026-08-22 15:45:00', 110.00, 13.20, 123.20, 'Efectivo', 7, 3, 4),
('FAC-060', '2026-09-05 10:05:00', 75.00, 9.00, 84.00, 'Tarjeta', 8, 4, 4),
('FAC-061', '2026-10-11 12:50:00', 180.00, 21.60, 201.60, 'Transferencia', 9, 5, 10),
('FAC-062', '2026-11-25 17:15:00', 300.00, 36.00, 336.00, 'Efectivo', 10, 3, 5),
('FAC-063', '2026-12-20 18:30:00', 400.00, 48.00, 448.00, 'Tarjeta', 11, 4, 6),
('FAC-064', '2026-12-25 19:00:00', 250.00, 30.00, 280.00, 'Transferencia', 12, 5, 6);

-- 2. Detalles de venta (Asignamos los productos a esas nuevas facturas)
-- Nota: Los IDs de venta asumen que continúan desde el 53 en adelante.
INSERT INTO detalle_venta (cantidad, precio_unitario, subtotal_linea, id_venta, id_producto) VALUES
(2, 60.00, 120.00, 53, 2),
(1, 80.00, 80.00, 54, 18),
(2, 75.00, 150.00, 55, 11),
(1, 60.00, 60.00, 56, 2),
(4, 50.00, 200.00, 57, 1),
(2, 45.00, 90.00, 58, 5),
(2, 55.00, 110.00, 59, 6),
(1, 75.00, 75.00, 60, 11),
(2, 90.00, 180.00, 61, 8),
(5, 50.00, 250.00, 62, 1),
```

```

SELECT
    m.NombreMes AS Mes,
    COUNT(v.id_venta) AS TotalVentas,
    COALESCE(SUM(v.total_factura), 0.00) AS MontoTotal
FROM (
    SELECT 1 AS MesNum, 'Enero' AS NombreMes UNION ALL
    SELECT 2, 'Febrero' UNION ALL
    SELECT 3, 'Marzo' UNION ALL
    SELECT 4, 'Abril' UNION ALL
    SELECT 5, 'Mayo' UNION ALL
    SELECT 6, 'Junio' UNION ALL
    SELECT 7, 'Julio' UNION ALL
    SELECT 8, 'Agosto' UNION ALL
    SELECT 9, 'Septiembre' UNION ALL
    SELECT 10, 'Octubre' UNION ALL
    SELECT 11, 'Noviembre' UNION ALL
    SELECT 12, 'Diciembre'
) AS m
LEFT JOIN ventas v ON m.MesNum = EXTRACT(MONTH FROM v.fecha_emision)
GROUP BY m.MesNum, m.NombreMes
ORDER BY m.MesNum;

```

Mes	TotalVentas	MontoTotal
Enero	1	134.40
Febrero	1	89.60
Marzo	1	168.00
Abril	1	67.20
Mayo	1	224.00
Junio	1	100.80
Julio	52	3337.30
Agosto	1	123.20
Septiembre	1	84.00
Octubre	1	201.60
Noviembre	1	336.00
Diciembre	2	728.00

Consulta 14. Compras acumuladas por cliente

```

SELECT c.nombres_completos AS Cliente,
COUNT(v.id_venta) AS CantidadCompras,
SUM(v.total_factura) AS MontoAcumulado
FROM clientes c
JOIN ventas v ON c.id_cliente =
v.id_cliente
GROUP BY c.id_cliente,
c.nombres_completos
ORDER BY MontoAcumulado DESC;

```

Cliente	CantidadCompras	MontoAcumulado
Cliente Once	3	609.00
Cliente Diez	3	454.45
Cliente Doce	3	421.45
Cliente Tres	3	375.00
Cliente Cinco	3	324.05
Cliente Ocho	3	279.50
Cliente Uno	3	278.15
Cliente Nueve	3	270.60
Cliente Dos	3	233.35
Cliente Seis	3	223.85
Cliente Siete	3	203.70
Cliente Trece	2	166.75
Cliente Veinte	2	166.75
Cliente Dieciocho	2	155.25
Cliente Cuatro	3	148.85
Cliente Quince	2	146.05
Cliente Catorce	2	110.40
Cliente Treinta	1	103.50
Cliente Dieciseis	2	100.05
Cliente Vinticinco	1	97.75
Cliente Diecisiete	2	86.25
Cliente Diecinueve	2	82.80
Cliente Vintiuno	1	74.75
Cliente Vinticuatro	1	69.00
Cliente Vintidos	1	66.70
Cliente Vintiocho	1	63.25
Cliente Vintitres	1	57.50
Cliente Treintaydos	1	55.20
Cliente Vintisiete	1	51.75
Cliente Vintinueve	1	46.00
Cliente Vintiseis	1	40.25
Cliente Treintayuno	1	32.20

Result 7 x

Consulta 15. Devoluciones por vendedor

```
SELECT e.nombres AS Vendedor,
COUNT(d.id_devolucion) AS
TotalDevoluciones
FROM devoluciones d
JOIN empleados e ON
d.id_empleado = e.id_empleado
GROUP BY e.id_empleado,
e.nombres
ORDER BY TotalDevoluciones
DESC;ORDER BY v.fecha_emision
DESC;
```

Result Grid			Filter Rows:
	Vendedor	TotalDevoluciones	
▶	Carlos	8	
	Ana	7	
	Luis	7	

Consulta 16. Clientes con compras superiores al promedio

```
SELECT
c.nombres_completos AS
Cliente,
SUM(v.total_factura) AS
TotalComprado
FROM clientes c
JOIN ventas v ON c.id_cliente =
v.id_cliente
GROUP BY c.id_cliente,
c.nombres_completos
HAVING SUM(v.total_factura) >
(SELECT AVG(total_factura) FROM
ventas);
```

Cliente	TotalComprado
▶ Cliente Uno	278.15
Cliente Dos	233.35
Cliente Tres	375.00
Cliente Cuatro	148.85
Cliente Cinco	324.05
Cliente Seis	223.85
Cliente Siete	203.70
Cliente Ocho	279.50
Cliente Nueve	270.60
Cliente Diez	454.45
Cliente Once	609.00
Cliente Doce	421.45
Cliente Trece	166.75
Cliente Catorce	110.40
Cliente Quince	146.05
Cliente Dieciseis	100.05
Cliente Dieciocho	155.25
Cliente Veinte	166.75
Cliente Vinticinco	97.75
Cliente Treinta	103.50

Result 9 ×

Consulta 17. Productos con precio superior al promedio

```
SELECT nombre_modelo AS
Producto, precio_venta
FROM productos
WHERE precio_venta > (SELECT
AVG(precio_venta) FROM
productos);
```

Producto	precio_venta
▶ Classic Low	60.00
Oxford Pro	85.00
Derby Classic	90.00
Trail Max	75.00
Loafer Comfort	65.00
Monk Strap	95.00
Chelsea Boot	80.00
Soccer Cleat	65.00
Moccasin Relax	58.00

Consulta 18. Vendedores con ventas superiores al promedio

```
SELECT
    e.nombres AS Vendedor,
    SUM(v.total_factura) AS
TotalVendido
FROM ventas v
JOIN empleados e ON v.id_empleado
= e.id_empleado
GROUP BY e.id_empleado, e.nombres
HAVING SUM(v.total_factura) >
(SELECT AVG(total_factura) FROM
ventas);
```

	Vendedor	TotalVendido
▶	Carlos	1839.55
	Ana	1927.75
	Luis	1826.80

Consulta 19. Productos que nunca se han vendido

Insercion de datos para ejemplo

```
INSERT INTO productos (sku, nombre_modelo, talla, color, material, costo_compra, precio_venta,
stock_actual, stock_minimo, id_categoria, id_proveedor) VALUES
('SKU998', 'Zapatillas Aburridas', 40.0, 'Gris', 'Sintetico', 15.00, 30.00, 50, 10, 2, 1),
('SKU999', 'Botas Pasadas de Moda', 38.0, 'Cafe', 'Sintetico', 20.00, 40.00, 30, 5, 2, 2);
```

```
• INSERT INTO productos (sku, nombre_modelo, talla, color, material, costo_compra, precio_venta, stock_actual, stock_minimo, id_categoria, id_proveedor) VALUES
('SKU998', 'Zapatillas Aburridas', 40.0, 'Gris', 'Sintetico', 15.00, 30.00, 50, 10, 2, 1),
('SKU999', 'Botas Pasadas de Moda', 38.0, 'Cafe', 'Sintetico', 20.00, 40.00, 30, 5, 2, 2);
```

```
SELECT
    id_producto,
    nombre_modelo,
    stock_actual
FROM productos
WHERE id_producto
NOT IN (SELECT
DISTINCT
id_producto FROM
detalle_venta);
```

Result Grid			
	id_producto	nombre_modelo	stock_actual
▶	23	Zapatillas Aburridas	50
	24	Botas Pasadas de Moda	30
•	NULL	NULL	NULL

Consulta 20. Clientes que nunca han realizado compras

Insertion de datos de ejemplo para la consulta

```
INSERT INTO clientes (tipo_identificacion, identificacion, nombres_completos, telefono, email, direccion_envio, ciudad) VALUES
('Cedula', '1799999990', 'Carlos Inactivo', '0990001111', 'carlos.inactivo@mail.com', 'Norte de Quito', 'Quito'),
('Cedula', '1799999991', 'Marta SinCompras', '0990002222', 'marta.sincompras@mail.com', 'Sur de Quito', 'Quito');
```

```
INSERT INTO clientes (tipo_identificacion, identificacion, nombres_completos, telefono, email, direccion_envio, ciudad) VALUES
('Cedula', '1799999990', 'Carlos Inactivo', '0990001111', 'carlos.inactivo@mail.com', 'Norte de Quito', 'Quito'),
('Cedula', '1799999991', 'Marta SinCompras', '0990002222', 'marta.sincompras@mail.com', 'Sur de Quito', 'Quito');
```

```
SELECT
    id_cliente,
    identificacion,
    nombres_completos,
    telefono
FROM clientes
WHERE id_cliente
NOT IN (SELECT
DISTINCT id_cliente
FROM ventas);
```

	id_cliente	identificacion	nombres_completos	telefono
▶	33	1799999990	Carlos Inactivo	0990001111
	34	1799999991	Marta SinCompras	0990002222
•	NULL	NULL	NULL	NULL

5.8 Vistas

Vista 1: Clientes Frecuentes

```
CREATE VIEW ClientesFrecuentes
AS
SELECT c.id_cliente,
c.nombres_completos, c.ciudad,
COUNT(v.id_venta) AS
TotalCompras,
SUM(v.total_factura) AS
MontoAcumulado
FROM clientes c
JOIN ventas v ON c.id_cliente
= v.id_cliente
GROUP BY c.id_cliente,
c.nombres_completos, c.ciudad
HAVING COUNT(v.id_venta) > 2
ORDER BY MontoAcumulado DESC;
SELECT * FROM
ClientesFrecuentes;
```

Result Grid					
Filter Rows:					
Export: Wrap Cell Content: IA					
	id_cliente	nombres_completos	ciudad	TotalCompras	MontoAcumulado
▶	11	Cliente Once	Guayaquil	3	609.00
	10	Cliente Diez	Quito	3	454.45
	12	Cliente Doce	Cuenca	3	421.45
	3	Cliente Tres	Cuenca	3	375.00
	5	Cliente Cinco	Quito	3	324.05
	8	Cliente Ocho	Manta	3	279.50
	1	Cliente Uno	Quito	3	278.15
	9	Cliente Nueve	Ibarra	3	270.60
	2	Cliente Dos	Guayaquil	3	233.35
	6	Cliente Seis	Loja	3	223.85
	7	Cliente Siete	Quito	3	203.70
	4	Cliente Cuatro	Ambato	3	148.85

Vista 2: Ventas Consolidadas

<pre> CREATE VIEW VentasConsolidadas AS SELECT v.id_venta, v.numero_factura, v.fecha_emision, c.nombres_completos AS Cliente, e.nombres AS Vendedor, v.metodo_pago, v.subtotal, v.iva, v.total_factura FROM ventas v JOIN clientes c ON v.id_cliente = c.id_cliente JOIN empleados e ON v.id_empleado = e.id_empleado; SELECT * FROM VentasConsolidadas; </pre>	Result Grid Filter Rows: Export: Wrap Cell Content:								
	id_venta	numero_factura	fecha_emision	Cliente	Vendedor	metodo_pago	subtotal	iva	total_factura
	1	FAC-001	2026-07-08 22:23:02	Cliente Uno	Carlos	Efectivo	50.00	7.50	57.50
	33	FAC-033	2026-07-08 22:23:02	Cliente Uno	Luis	Efectivo	75.00	11.25	86.25
	53	FAC-053	2026-01-15 10:30:00	Cliente Uno	Carlos	Efectivo	120.00	14.40	134.40
	2	FAC-002	2026-07-08 22:23:02	Cliente Dos	Ana	Tarjeta	60.00	9.00	69.00
	34	FAC-034	2026-07-08 22:23:02	Cliente Dos	Carlos	Tarjeta	65.00	9.75	74.75
	54	FAC-054	2026-02-20 14:15:00	Cliente Dos	Ana	Tarjeta	80.00	9.60	89.60
	3	FAC-003	2026-07-08 22:23:02	Cliente Tres	Luis	Efectivo	85.00	12.75	97.75
	35	FAC-035	2026-07-08 22:23:02	Cliente Tres	Ana	Efectivo	95.00	14.25	109.25
	55	FAC-055	2026-03-10 09:45:00	Cliente Tres	Luis	Transferencia	150.00	18.00	168.00
	4	FAC-004	2026-07-08 22:23:02	Cliente Cuatro	Carlos	Transferencia	35.00	5.25	40.25
	36	FAC-036	2026-07-08 22:23:02	Cliente Cuatro	Luis	Transferencia	36.00	5.40	41.40
	56	FAC-056	2026-04-05 16:20:00	Cliente Cuatro	Carlos	Efectivo	60.00	7.20	67.20
	5	FAC-005	2026-07-08 22:23:02	Cliente Cinco	Ana	Efectivo	45.00	6.75	51.75
	37	FAC-037	2026-07-08 22:23:02	Cliente Cinco	Carlos	Efectivo	42.00	6.30	48.30
	57	FAC-057	2026-05-12 11:10:00	Cliente Cinco	Ana	Tarjeta	200.00	24.00	224.00
	6	FAC-006	2026-07-08 22:23:02	Cliente Seis	Luis	Tarjeta	55.00	8.25	63.25
	38	FAC-038	2026-07-08 22:23:02	Cliente Seis	Ana	Tarjeta	52.00	7.80	59.80
	58	FAC-058	2026-06-18 13:30:00	Cliente Seis	Luis	Transferencia	90.00	10.80	100.80
	7	FAC-007	2026-07-08 22:23:02	Cliente Siete	Carlos	Efectivo	40.00	6.00	46.00

Vista 3: Productos con Bajo Stock

<pre> CREATE VIEW ProductosBajoStock AS SELECT p.id_producto, p.nombre_modelo, c.nombre AS Categoria, p.stock_actual, p.stock_minimo FROM productos p JOIN categorias c ON p.id_categoria = c.id_categoria WHERE p.stock_actual <= p.stock_minimo; SELECT * FROM ProductosBajoStock; </pre>	Result Grid Filter Rows: Export: Wrap Cell Content:				
	id_producto	nombre_modelo	Categoria	stock_actual	stock_minimo
	3	Oxford Pro	Formal	4	10
	7	Sneaker High	Casual	8	15

Vista 4: Promociones Aplicadas

CREATE VIEW PromocionesAplicadas AS
SELECT v.id_venta,
v.numero_factura,
c.nombres_completos AS Cliente,
p.codigo_promo AS Promocion,
p.tipo_descuento,
p.valor_descuento,
v.subtotal,
v.total_factura
FROM ventas v
JOIN clientes c ON v.id_cliente =
c.id_cliente
LEFT JOIN promociones p ON
v.id_promocion = p.id_promocion
WHERE v.id_promocion IS NOT NULL;
SELECT * FROM
PromocionesAplicadas;

Result Grid		Filter Rows:		Exports	Wrap Cell Contents:			
	id_venta	numero_factura	Cliente	Promocion	tipo_descuento	valor_descuento	subtotal	total_factura
▶	2	FAC-002	Cliente Dos	VERANO	Porcentaje	15.00	60.00	69.00
	4	FAC-004	Cliente Cuatro	MADRES	Porcentaje	20.00	35.00	40.25
	8	FAC-008	Cliente Ocho	PADRES	Fijo	5.00	90.00	103.50
	12	FAC-012	Cliente Doce	REGRESO	Porcentaje	10.00	65.00	74.75
	16	FAC-016	Cliente Dieciseis	BLACKFRI	Porcentaje	50.00	52.00	59.80
	20	FAC-020	Cliente Veinte	NAVIDAD	Porcentaje	25.00	55.00	63.25
	24	FAC-024	Cliente Vinticuatro	LIQUIDA	Fijo	10.00	60.00	69.00
	28	FAC-028	Cliente Vintocho	CUMPLE	Fijo	15.00	55.00	63.25
	32	FAC-032	Cliente Treintaydos	MAYORISTA	Porcentaje	12.00	48.00	55.20
	36	FAC-036	Cliente Cuatro	FLASH	Porcentaje	30.00	36.00	41.40
	40	FAC-040	Cliente Ocho	INVIERNO	Porcentaje	15.00	80.00	92.00
	44	FAC-044	Cliente Doce	PRIMAVERA	Porcentaje	10.00	58.00	66.70
	48	FAC-048	Cliente Dieciseis	VERANO	Porcentaje	15.00	35.00	40.25
	52	FAC-052	Cliente Veinte	MADRES	Porcentaje	20.00	90.00	103.50
	53	FAC-053	Cliente Uno	LIQUIDA	Fijo	10.00	120.00	134.40
	55	FAC-055	Cliente Tres	INVIERNO	Porcentaje	15.00	150.00	168.00
	56	FAC-056	Cliente Cuatro	PRIMAVERA	Porcentaje	10.00	60.00	67.20
	57	FAC-057	Cliente Cinco	MADRES	Porcentaje	20.00	200.00	224.00
	58	FAC-058	Cliente Seis	VERANO	Porcentaje	15.00	90.00	100.80
	59	FAC-059	Cliente Siete	REGRESO	Porcentaje	10.00	110.00	123.20
	60	FAC-060	Cliente Ocho	REGRESO	Porcentaje	10.00	75.00	84.00
	61	FAC-061	Cliente Nueve	FLASH	Porcentaje	30.00	180.00	201.60
	62	FAC-062	Cliente Diez	BLACKFRI	Porcentaje	50.00	300.00	336.00

PromocionesAplicadas 20

Vista 5: Devoluciones Detalladas

```
CREATE VIEW
DevolucionesDetalladas AS
SELECT d.id_devolucion,
d.fecha_devolucion,
c.nombres_completos AS
Cliente, p.nombre_modelo
AS Producto,
d.cantidad_devuelta,
d.motivo,
d.estado_calzado,
e.nombres AS
RegistradoPor
FROM devoluciones d
JOIN ventas v ON
d.id_venta = v.id_venta
JOIN clientes c ON
v.id_cliente =
c.id_cliente
JOIN productos p ON
d.id_producto =
p.id_producto
JOIN empleados e ON
d.id_empleado =
e.id_empleado;
SELECT * FROM
DevolucionesDetalladas;
```

	id_devolucion	fecha_devolucion	Cliente	Producto	cantidad_devuelta	motivo	estado_calzado	RegistradoPor
▶	1	2026-07-08 22:23:02	Cliente Uno	Runner X	1	Talla incorrecta	Nuevo	Carlos
	2	2026-07-08 22:23:02	Cliente Dos	Classic Low	1	Defecto de fabrica	Defectuoso	Ana
	3	2026-07-08 22:23:02	Cliente Tres	Oxford Pro	1	No le gusto	Nuevo	Luis
	4	2026-07-08 22:23:02	Cliente Cuatro	Colegial Basic	1	Talla incorrecta	Nuevo	Carlos
	5	2026-07-08 22:23:02	Cliente Cinco	Heels Elegance	1	Mancha en el cuero	Defectuoso	Ana
	6	2026-07-08 22:23:02	Cliente Seis	Runner Pro	1	Suela despegada	Defectuoso	Luis
	7	2026-07-08 22:23:02	Cliente Siete	Sneaker High	1	Cambio por otro color	Nuevo	Carlos
	8	2026-07-08 22:23:02	Cliente Ocho	Derby Classic	1	Talla incorrecta	Nuevo	Ana
	9	2026-07-08 22:23:02	Cliente Nueve	Kids Play	1	Defecto costura	Defectuoso	Luis
	10	2026-07-08 22:23:02	Cliente Diez	Platform Star	1	No le gusto	Nuevo	Carlos
	11	2026-07-08 22:23:02	Cliente Once	Trail Max	1	Talla incorrecta	Nuevo	Ana
	12	2026-07-08 22:23:02	Cliente Doce	Loafer Comfort	1	Defecto de fabrica	Defectuoso	Luis
	13	2026-07-08 22:23:02	Cliente Trece	Monk Strap	1	Cambio modelo	Nuevo	Carlos
	14	2026-07-08 22:23:02	Cliente Catorce	School Girl	1	Talla incorrecta	Nuevo	Ana
	15	2026-07-08 22:23:02	Cliente Quince	Wedge Summer	1	Mancha	Defectuoso	Luis
	16	2026-07-08 22:23:02	Cliente Dieciseis	Marathon Lite	1	No le gusto	Nuevo	Carlos
	17	2026-07-08 22:23:02	Cliente Diecisi...	Slip-On Easy	1	Cambio por talla	Nuevo	Ana
	18	2026-07-08 22:23:02	Cliente Diecio...	Chelsea Boot	1	Cordon roto	Defectuoso	Luis
	19	2026-07-08 22:23:02	Cliente Diecin...	Sport Junior	1	Talla incorrecta	Nuevo	Carlos
	20	2026-07-08 22:23:02	Cliente Veinte	Stiletto Chic	1	Defecto de fabrica	Defectuoso	Ana
	21	2026-07-08 22:23:02	Cliente Vintiuno	Soccer Cleat	1	No le gusto	Nuevo	Luis
	22	2026-07-08 22:23:02	Cliente Vintidos	Moccasin Relax	1	Cambio por talla	Nuevo	Carlos

Vista 6: Desempeño de Vendedores

```
CREATE VIEW DesempenoVendedores
AS
SELECT e.id_empleado,
e.nombres, e.apellidos,
COUNT(v.id_venta) AS
TotalVentas,
SUM(v.total_factura) AS
MontoGenerado,
COUNT(d.id_devolucion) AS
TotalDevoluciones
FROM empleados e
LEFT JOIN ventas v ON
e.id_empleado = v.id_empleado
LEFT JOIN devoluciones d ON
e.id_empleado = d.id_empleado
GROUP BY e.id_empleado,
e.nombres, e.apellidos;
SELECT * FROM
DesempenoVendedores;
```

Result Grid Filter Rows: Export: Wrap Cell Content:						
	id_empleado	nombres	apellidos	TotalVentas	MontoGenerado	TotalDevoluciones
▶	1	Juan	Perez	0	NULL	0
	2	Maria	Jose	0	NULL	0
	3	Carlos	Mendoza	176	14716.40	176
	4	Ana	Torres	147	13494.25	147
	5	Luis	Paredes	147	12787.60	147
	6	Sofia	Rios	0	NULL	0
	7	Miguel	Ortega	0	NULL	0
	8	Elena	Castro	0	NULL	0
	9	Andres	Vega	0	NULL	0
	10	Lucia	Gomez	0	NULL	0
	11	Pedro	Luna	0	NULL	0
	12	Marta	Diaz	0	NULL	0

5.9 Funciones

1. Calcular IVA: Retorna el valor del IVA sobre un monto dado

```
DELIMITER //  
CREATE FUNCTION  
calcular_iva(monto  
DECIMAL(10,2))  
RETURNS  
DECIMAL(10,2)  
DETERMINISTIC  
BEGIN  
    RETURN monto *  
    0.15;  
END //  
DELIMITER ;
```

```
SELECT calcular_iva(100) AS iva_calculado;
```



The screenshot shows a SQL query result grid with the following data:

	iva_calculado
▶	15.00

2. Calcular Descuento: Calcula el descuento según porcentaje o promoción

```
DELIMITER //  
CREATE FUNCTION  
calcular_descuento(monto  
DECIMAL(10,2), porcentaje  
DECIMAL(5,2))  
RETURNS DECIMAL(10,2)  
DETERMINISTIC  
BEGIN  
    RETURN monto *  
    (porcentaje / 100);  
END //  
DELIMITER ;
```

```
SELECT calcular_descuento(50, 20) AS descuento_calculado;
```



The screenshot shows a SQL query result grid with the following data:

	descuento_calculado
▶	10.00

3. Calcular Edad: Determina la edad de un cliente a partir de su fecha de nacimiento

```
DELIMITER //  
CREATE FUNCTION  
calcular_edad(fecha_nacimiento  
DATE)  
RETURNS INT  
DETERMINISTIC  
BEGIN  
    -- TIMESTAMPDIFF calcula los  
    años exactos transcurridos hasta  
    hoy  
    RETURN TIMESTAMPDIFF(YEAR,  
    fecha_nacimiento, CURDATE());  
END //  
DELIMITER ;
```

```
SELECT calcular_edad('1995-05-15') AS edad_cliente; SELECT *  
FROM DesempenoVendedores;
```



The screenshot shows a SQL query result grid with the following data:

	edad_cliente
▶	31

4. Calcular Comisión: Calcula la comisión de un vendedor según sus ventas

```
DELIMITER //
CREATE FUNCTION calcular_comision(monto_ventas
DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE comision DECIMAL(10,2);

    IF monto_ventas >= 1000 THEN
        SET comision = monto_ventas * 0.05; -- 5%
    ELSE
        SET comision = monto_ventas * 0.02; -- 2%
    END IF;

    RETURN comision;
END //
DELIMITER ;
```

```
SELECT calcular_comision(1500) AS
comision_ganada;
```

comision_ganada
75.00

Justificación: En MySQL, las funciones (FUNCTION) solo pueden devolver un único valor escalar (número, texto o fecha).

A diferencia de PostgreSQL o SQL Server, es técnicamente imposible que una función en MySQL devuelva una tabla con filas y columnas. Si se necesita que el motor devuelva un listado completo, es obligatorio utilizar un procedimiento almacenado (PROCEDURE).

5. Productos Bajo Stock: Retorna listado de productos con stock menor al mínimo

```
DELIMITER //
CREATE PROCEDURE
productos_bajo_stock()
BEGIN
    SELECT id_producto,
    nombre_modelo, stock_actual,
    stock_minimo
    FROM productos
    WHERE stock_actual <
    stock_minimo;
END //
DELIMITER ;
```

```
CALL productos_bajo_stock();
```

id_producto	nombre_modelo	stock_actual	stock_minimo
3	Oxford Pro	4	10
7	Sneaker High	8	15

6. Clientes Frecuentes: Identifica clientes según número de compras o monto acumulado

```
DELIMITER //
CREATE PROCEDURE clientes_frecuentes(IN monto_minimo
DECIMAL(10,2))
BEGIN
    SELECT c.id_cliente,
    CONCAT(c.nombres, ' ', c.apellidos) AS
    nombre_completo,
    SUM(v.total_factura) AS total_comprado
    FROM clientes c
    JOIN ventas v ON c.id_cliente = v.id_cliente
    GROUP BY c.id_cliente, c.nombres, c.apellidos
    HAVING SUM(v.total_factura) >= monto_minimo
    ORDER BY total_comprado DESC;
END //
DELIMITER ;
```

```
CALL clientes_frecuentes(500);
```

id_producto	nombre_modelo	stock_actual	stock_minimo
3	Oxford Pro	4	10
7	Sneaker High	8	15

5.10 Procedimientos Almacenados

1. Registrar Cliente: identificación, nombres, apellidos, teléfono, correo → valida duplicados

```
DELIMITER //
CREATE PROCEDURE
registrar_cliente(
    IN p_tipo_id
    VARCHAR(20),
    IN p_identificacion
    VARCHAR(15),
    IN
    p_nombres_completos
    VARCHAR(150),
    IN p_telefono
    VARCHAR(20),
    IN p_email
    VARCHAR(100)
)
BEGIN
    DECLARE existe INT;
    SELECT COUNT(*) INTO
    existe FROM clientes
    WHERE identificacion =
    p_identificacion;

    IF existe > 0 THEN
        SIGNAL SQLSTATE
        '45000' SET MESSAGE_TEXT
        = 'Error: El cliente ya
        se encuentra
        registrado.';
    ELSE
        INSERT INTO
        clientes
        (tipo_identificacion,
        identificacion,
        nombres_completos,
        telefono, email)
        VALUES
        (p_tipo_id,
        p_identificacion,
        p_nombres_completos,
        p_telefono, p_email);
    END IF;
END //
DELIMITER ;
```

Tabla Original:

	id_cliente	tipo_identificacion	identificacion	nombres_completos	telefono	email	direccion_envio	ciudad	tipo_cliente
	19	Cedula	1719191919	Cliente Diecinueve	0991919191	c19@mail.com	Dir 19	Quito	Minorista
	20	Cedula	1720202020	Cliente Veinte	0992020202	c20@mail.com	Dir 20	Guayaquil	Minorista
	21	Cedula	1721212121	Cliente Vintiuno	0992121212	c21@mail.com	Dir 21	Cuenca	Minorista
	22	Cedula	1723232323	Cliente Vintidos	0992323232	c22@mail.com	Dir 22	Ambato	Minorista
	23	Cedula	1724242424	Cliente Vintitres	0992424242	c23@mail.com	Dir 23	Quito	Minorista
	24	Cedula	1725252525	Cliente Vinticuatro	0992525252	c24@mail.com	Dir 24	Loja	Minorista
	25	Cedula	1726262626	Cliente Vinticinco	0992626262	c25@mail.com	Dir 25	Quito	Minorista
	26	Cedula	1727272727	Cliente Vintiseis	0992727272	c26@mail.com	Dir 26	Manta	Minorista
	27	Cedula	1728282828	Cliente Vintisiete	0992828282	c27@mail.com	Dir 27	Ibarra	Minorista
	28	Cedula	1729292929	Cliente Vintiocho	0992929292	c28@mail.com	Dir 28	Quito	Minorista
	29	Cedula	1730303030	Cliente Vintinueve	0993030303	c29@mail.com	Dir 29	Guayaquil	Minorista
	30	Cedula	1731313131	Cliente Treinta	0993131313	c30@mail.com	Dir 30	Cuenca	Minorista
	31	Cedula	1732323232	Cliente Treintayuno	0993232323	c31@mail.com	Dir 31	Ambato	Minorista
	32	Cedula	1735353535	Cliente Treintaydos	0993535353	c32@mail.com	Dir 32	Quito	Minorista
	33	Cedula	1799999990	Carlos Inactivo	0990001111	carlos.inactiv...	Norte de Quito	Quito	Minorista
	34	Cedula	1799999991	Marta SinCompras	0990002222	marta.sincom...	Sur de Quito	Quito	Minorista
	35	Cedula	1755125555	Pedro Martinez	0988888888	pedro@email...			Minorista
	36	Cedula	1745655555	Pedro Martinez	0988888888	pedro@email...			Minorista

Captura de ejecucion Correcta:

CALL registrar_cliente('Cedula', '1745655145', 'Pedro Martinez', '0988888888', 'pedro@email.com');

```
29 18:52:10 CREATE PROCEDURE registrar_cliente( IN p_tipo_id VARCHAR(20), IN p_identificacion VARC... 0 row(s) affected
30 18:53:50 CALL registrar_cliente('Cedula', '1745655555', 'Pedro Martinez', '0988888888', 'pedro@email.com') 1 row(s) affected
```

Tabla con los cambios del procedimiento:

SELECT * FROM clientes WHERE identificación = '1745655145';

	id_cliente	tipo_identificacion	identificacion	nombres_completos	telefono	email	direccion_envio	ciudad	tipo_cliente
	37	Cedula	1745655145	Pedro Martinez	0988888888	pedro@email.com			Minorista

2. Registrar Producto: nombre, precio, stock, tipoProducto → nacional o importado

```

DELIMITER //
CREATE PROCEDURE
registrar_producto(
    IN p_sku
    VARCHAR(50),
    IN p_nombre
    VARCHAR(150),
    IN p_talla
    DECIMAL(4,1),
    IN p_costo
    DECIMAL(10,2),
    IN p_precio
    DECIMAL(10,2),
    IN p_stock INT,
    IN
    p_id_categoria INT,
    IN
    p_id_proveedor INT
)
BEGIN
    DECLARE v_pais
    VARCHAR(50);
    DECLARE v_tipo
    VARCHAR(20);

    SELECT
    pais_origen INTO
    v_pais FROM
    proveedores WHERE
    id_proveedor =
    p_id_proveedor;

    IF v_pais =
    'Ecuador' THEN
        SET v_tipo =
        'Nacional';
    ELSE
        SET v_tipo =
        'Importado';
    END IF;

    INSERT INTO
    productos (sku,
    nombre_modelo,
    talla, costo_compra,
    precio_venta,
    stock_actual,
    id_categoria,
    id_proveedor)
    VALUES (p_sku,
    CONCAT(p_nombre, '
    (' , v_tipo, ')'),
    p_talla, p_costo,
    p_precio, p_stock,
    p_id_categoria,
    p_id_proveedor);
END //
DELIMITER ;
    
```

Tabla Original:

	id_producto	sku	nombre_modelo	talla	color	material	costo_compra	precio_venta	stock_actual	stock_minimo	id_categoria	id_proveedor
	17	SKU017	Slip-On Easy	42.5	Gris	Lona	15.00	30.00	140	20	2	4
	18	SKU018	Chelsea Boot	40.0	Negro	Gamuza	38.00	80.00	65	10	3	8
	19	SKU019	Sport Junior	30.0	Rojo	Sintetico	14.00	32.00	180	25	4	1
	20	SKU020	Stiletto Chic	35.5	Rojo	Charol	25.00	55.00	50	10	5	5
	21	SKU021	Soccer Cleat	41.0	Blanco	Sintetico	30.00	65.00	75	10	1	7
	22	SKU022	Moccasin Relax	39.0	Cafe	Cuero	28.00	58.00	80	10	2	8
	23	SKU998	Zapatillas Abur...	40.0	Gris	Sintetico	15.00	30.00	50	10	2	1
	24	SKU999	Botas Pasadas ...	38.0	Cafe	Sintetico	20.00	40.00	30	5	2	2

Captura de ejecucion Correcta:

CALL registrar_producto('SKU-999', 'Oxford Ejecutivo', 42.0, 30.00, 65.00, 50, 1, 1);

✓	34	19:01:25	CREATE PROCEDURE registrar_producto(IN p_sku VARCHAR(50). IN p_nombre VARCHAR(1...	0 row(s) affected
✓	35	19:01:35	CALL registrar_producto('SKU-999', 'Oxford Ejecutivo', 42.0, 30.00, 65.00, 50, 1, 1)	1 row(s) affected

Tabla con los cambios del procedimiento:

SELECT * FROM productos WHERE sku = 'SKU-999';

	id_producto	sku	nombre_modelo	talla	color	material	costo_compra	precio_venta	stock_actual	stock_minimo	id_categoria	id_proveedor
▶	25	SKU-999	Oxford Ejecutivo (Nacional)	42.0	NULL	NULL	30.00	65.00	50	10	1	1
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

3. Registrar Venta: Registra venta + actualiza inventario + aplica promociones (ventas, detalle_venta, inventario)

```
DELIMITER //
CREATE PROCEDURE registrar_venta(
    IN p_factura VARCHAR(50),
    IN p_id_cliente INT,
    IN p_id_empleado INT,
    IN p_id_producto INT,
    IN p_cantidad INT,
    IN p_metodo_pago VARCHAR(50)
)
BEGIN
    DECLARE v_precio DECIMAL(10,2);
    DECLARE v_stock INT;
    DECLARE v_subtotal DECIMAL(10,2);
    DECLARE v_iva DECIMAL(10,2);
    DECLARE v_total DECIMAL(10,2);
    DECLARE v_id_venta INT;

    SELECT precio_venta, stock_actual INTO v_precio, v_stock FROM productos WHERE id_producto =
p_id_producto;

    IF v_stock >= p_cantidad THEN
        SET v_subtotal = v_precio * p_cantidad;
        SET v_iva = v_subtotal * 0.15;
        SET v_total = v_subtotal + v_iva;

        INSERT INTO ventas (numero_factura, subtotal, iva, total_factura, metodo_pago,
id_cliente, id_empleado)
        VALUES (p_factura, v_subtotal, v_iva, v_total, p_metodo_pago, p_id_cliente,
p_id_empleado);

        SET v_id_venta = LAST_INSERT_ID();

        INSERT INTO detalle_venta (cantidad, precio_unitario, subtotal_linea, id_venta,
id_producto)
        VALUES (p_cantidad, v_precio, v_subtotal, v_id_venta, p_id_producto);

        UPDATE productos SET stock_actual = stock_actual - p_cantidad WHERE id_producto =
p_id_producto;

        INSERT INTO inventario (tipo_movimiento, cantidad, documento_referencia, id_producto,
id_empleado)
        VALUES ('SALIDA VENTA', p_cantidad, p_factura, p_id_producto, p_id_empleado);
    ELSE
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Stock insuficiente para realizar la
venta.';
    END IF;
END //
DELIMITER ;
CALL registrar_venta('FAC-999', 1, 1, 1, 2, 'Efectivo');
```

Capturas de correcta implementacion de procedimiento

Tabla Original:

SELECT * FROM ventas;

Result Grid											
Filter Rows:			Edit:			Export/Import:			Wrap Cell Content:		
id_venta	numero_factura	fecha_emision	subtotal	iva	total_factura	metodo_pago	estado_venta	id_cliente	id_empleado	id_promocion	
58	FAC-058	2026-06-18 13:30:00	90.00	10.80	100.80	Transferencia	Completada	6	5	1	
59	FAC-059	2026-08-22 15:45:00	110.00	13.20	123.20	Efectivo	Completada	7	3	4	
60	FAC-060	2026-09-05 10:05:00	75.00	9.00	84.00	Tarjeta	Completada	8	4	4	
61	FAC-061	2026-10-11 12:50:00	180.00	21.60	201.60	Transferencia	Completada	9	5	10	
62	FAC-062	2026-11-25 17:15:00	300.00	36.00	336.00	Efectivo	Completada	10	3	5	
63	FAC-063	2026-12-20 18:30:00	400.00	48.00	448.00	Tarjeta	Completada	11	4	6	
64	FAC-064	2026-12-23 19:00:00	250.00	30.00	280.00	Transferencia	Completada	12	5	6	
* NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	

SELECT * FROM inventario;

id_movimiento	tipo_movimiento	cantidad	fecha_movimiento	documento_referencia	id_producto	id_empleado
16	Entrada	105	2026-07-08 22:23:02	Factura Proveedor 16	16	2
17	Entrada	140	2026-07-08 22:23:02	Factura Proveedor 17	17	2
18	Entrada	65	2026-07-08 22:23:02	Factura Proveedor 18	18	2
19	Entrada	180	2026-07-08 22:23:02	Factura Proveedor 19	19	2
20	Entrada	50	2026-07-08 22:23:02	Factura Proveedor 20	20	2
21	Entrada	75	2026-07-08 22:23:02	Factura Proveedor 21	21	2
22	Entrada	80	2026-07-08 22:23:02	Factura Proveedor 22	22	2
* NULL	NULL	NULL	NULL	NULL	NULL	NULL

Captura de ejecucion Correcta:

✓	5	19:16:28	CREATE PROCEDURE registrar_venta(IN p_factura VARCHAR(50), IN p_id_cliente INT, IN...	0 row(s) affected
✓	6	19:16:28	CALL registrar_venta('FAC-997', 1, 1, 1, 2, 'Efectivo')	1 row(s) affected

Tabla con los cambios del procedimiento:

SELECT * FROM ventas WHERE numero_factura = 'FAC-997';

id_venta	numero_factura	fecha_emision	subtotal	iva	total_factura	metodo_pago	estado_venta	id_cliente	id_empleado	id_promocion
67	FAC-997	2026-07-12 19:16:28	100.00	15.00	115.00	Efectivo	Completada	1	1	NULL
* NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

SELECT * FROM inventario WHERE documento_referencia = 'FAC-997';

id_movimiento	tipo_movimiento	cantidad	fecha_movimiento	documento_referencia	id_producto	id_empleado
24	SALIDA VENTA	2	2026-07-12 19:16:28	FAC-997	1	1
* NULL	NULL	NULL	NULL	NULL	NULL	NULL

4. Registrar Devolución: Registra devolución e incrementa stock automáticamente

```

DELIMITER //
CREATE PROCEDURE
registrar_devolucion(
    IN p_id_venta
    INT,
    IN p_id_producto
    INT,
    IN p_cantidad
    INT,
    IN p_motivo
    VARCHAR(200),
    IN
    p_estado_calzado
    VARCHAR(50),
    IN p_id_empleado
    INT
)
BEGIN
    INSERT INTO
    devoluciones (motivo,
    cantidad_devuelta,
    estado_calzado,
    id_venta,
    id_producto,
    id_empleado)
    VALUES (p_motivo,
    p_cantidad,
    p_estado_calzado,
    p_id_venta,
    p_id_producto,
    p_id_empleado);

    UPDATE productos
    SET stock_actual =
    stock_actual +
    p_cantidad WHERE
    id_producto =
    p_id_producto;

    INSERT INTO
    inventario
    (tipo_movimiento,
    cantidad,
    documento_referencia,
    id_producto,
    id_empleado)
    VALUES ('ENTRADA
    DEVOLUCION',
    p_cantidad,
    CONCAT('Devolucion
    Factura ',
    p_id_venta),
    p_id_producto,
    p_id_empleado);
END //
DELIMITER ;
    
```

Tabla Original:

	id_devolucion	fecha_devolucion	motivo	cantidad_devuelta	estado_calzado	id_venta	id_producto	id_empleado
	13	2026-07-08 22:23:02	Cambio modelo	1	Nuevo	13	13	3
	14	2026-07-08 22:23:02	Talla incorrecta	1	Nuevo	14	14	4
	15	2026-07-08 22:23:02	Mancha	1	Defectuoso	15	15	5
	16	2026-07-08 22:23:02	No le gusto	1	Nuevo	16	16	3
	17	2026-07-08 22:23:02	Cambio por talla	1	Nuevo	17	17	4
	18	2026-07-08 22:23:02	Cordon roto	1	Defectuoso	18	18	5
	19	2026-07-08 22:23:02	Talla incorrecta	1	Nuevo	19	19	3
	20	2026-07-08 22:23:02	Defecto de fabrica	1	Defectuoso	20	20	4
	21	2026-07-08 22:23:02	No le gusto	1	Nuevo	21	21	5
	22	2026-07-08 22:23:02	Cambio por talla	1	Nuevo	22	22	3
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Captura de ejecucion Correcta:

CALL registrar_devolucion(1, 1, 1, 'Talla incorrecta', 'Nuevo', 1);

 10 19:24:00 CREATE PROCEDURE registrar_devolucion(IN p_id_venta INT, IN p_id_producto INT, IN ... 0 row(s) affected
 11 19:24:00 CALL registrar_devolucion(1, 1, 1, 'Talla incorrecta', 'Nuevo', 1) 1 row(s) affected

Tabla con los cambios del procedimiento:

SELECT * FROM devoluciones ORDER BY id_devolucion DESC LIMIT 1;

	id_devolucion	fecha_devolucion	motivo	cantidad_devuelta	estado_calzado	id_venta	id_producto	id_empleado
	24	2026-07-12 19:24:23	Talla incorrecta	1	Nuevo	1	1	1
»	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

5. Aplicar Promoción: Verifica condiciones y calcula descuento

```
DELIMITER //
CREATE PROCEDURE aplicar_promocion(
    IN p_codigo_promo VARCHAR(20),
    IN p_monto_original DECIMAL(10,2)
)
BEGIN
    DECLARE v_descuento DECIMAL(10,2);
    DECLARE v_tipo VARCHAR(20);
    DECLARE v_monto_final DECIMAL(10,2);

    SELECT valor_descuento, tipo_descuento INTO v_descuento, v_tipo
    FROM promociones
    WHERE codigo_promo = p_codigo_promo AND CURDATE() BETWEEN fecha_inicio AND fecha_fin;

    IF v_descuento IS NOT NULL THEN
        IF v_tipo = 'Porcentaje' THEN
            SET v_monto_final = p_monto_original - (p_monto_original * (v_descuento / 100));
        ELSE
            SET v_monto_final = p_monto_original - v_descuento;
        END IF;
        SELECT p_monto_original AS Monto_Original, v_monto_final AS Monto_Con_Descuento, 'Promocion
Aplicada' AS Estado;
    ELSE
        SELECT p_monto_original AS Monto_Original, p_monto_original AS Monto_Con_Descuento,
        'Promocion Invalida o Expirada' AS Estado;
    END IF;
END //
DELIMITER ;
```

Captura de ejecucion Correcta:

```
CALL registrar_devolucion(1, 1, 1, 'Talla incorrecta', 'Nuevo', 1);
```

```
10 19:24:00 CREATE PROCEDURE registrar_devolucion( IN p_id_venta INT, IN p_id_producto INT, IN ... 0 row(s) affected
11 19:24:00 CALL registrar_devolucion(1, 1, 1, 'Talla incorrecta', 'Nuevo', 1) 1 row(s) affected
```

Tabla con los cambios del procedimiento:

```
SELECT codigo_promo, tipo_descuento, valor_descuento, fecha_inicio, fecha_fin
FROM promociones
WHERE codigo_promo = 'VERANO';
```

	codigo_promo	tipo_descuento	valor_descuento	fecha_inicio	fecha_fin
▶	VERANO	Porcentaje	15.00	2026-06-01	2026-08-31

6. Calcular Ventas Mensuales: Muestra Mes y TotalVentas

```

DROP PROCEDURE IF EXISTS calcular_ventas_mensuales;
DELIMITER //
CREATE PROCEDURE calcular_ventas_mensuales(IN p_anio INT)
BEGIN

    SELECT MONTH(fecha_emision) AS Mes, SUM(total_factura) AS TotalVentas
    FROM ventas
    WHERE YEAR(fecha_emision) = p_anio
    GROUP BY MONTH(fecha_emision)
    ORDER BY Mes;

END //
DELIMITER ;

```

Captura de ejecucion Correcta:

CALL calcular_ventas_mensuales(2026);

✓	16	20:11:07	CREATE PROCEDURE calcular_ventas_mensuales(IN p_anio INT) BEGIN -- Este SELECT intern...	0 row(s) affected
✓	17	20:11:42	CALL calcular_ventas_mensuales(2026)	12 row(s) returned

Tabla con los cambios del procedimiento:

```

SELECT numero_factura, fecha_emision, total_factura, estado_venta
FROM ventas
WHERE YEAR(fecha_emision) = 2026;

```

	numero_factura	fecha_emision	total_factura	estado_venta		numero_factura	fecha_emision	total_factura	estado_venta
▶	FAC-001	2026-07-08 22:23:02	57.50	Completada		FAC-028	2026-07-08 22:23:02	63.25	Completada
	FAC-002	2026-07-08 22:23:02	69.00	Completada		FAC-029	2026-07-08 22:23:02	46.00	Completada
	FAC-003	2026-07-08 22:23:02	97.75	Completada		FAC-030	2026-07-08 22:23:02	103.50	Completada
	FAC-004	2026-07-08 22:23:02	40.25	Completada		FAC-031	2026-07-08 22:23:02	32.20	Completada
	FAC-005	2026-07-08 22:23:02	51.75	Completada		FAC-032	2026-07-08 22:23:02	55.20	Completada
	FAC-006	2026-07-08 22:23:02	63.25	Completada		FAC-033	2026-07-08 22:23:02	86.25	Completada
	FAC-007	2026-07-08 22:23:02	46.00	Completada		FAC-034	2026-07-08 22:23:02	74.75	Completada
	FAC-008	2026-07-08 22:23:02	103.50	Completada		FAC-035	2026-07-08 22:23:02	109.25	Completada
	FAC-009	2026-07-08 22:23:02	32.20	Completada		FAC-036	2026-07-08 22:23:02	41.40	Completada
	FAC-010	2026-07-08 22:23:02	55.20	Completada		FAC-037	2026-07-08 22:23:02	48.30	Completada
	FAC-011	2026-07-08 22:23:02	86.25	Completada		FAC-038	2026-07-08 22:23:02	59.80	Completada
	FAC-012	2026-07-08 22:23:02	74.75	Completada		FAC-039	2026-07-08 22:23:02	34.50	Completada
	FAC-013	2026-07-08 22:23:02	109.25	Completada		FAC-040	2026-07-08 22:23:02	92.00	Completada
	FAC-014	2026-07-08 22:23:02	41.40	Completada		FAC-041	2026-07-08 22:23:02	36.80	Completada
	FAC-015	2026-07-08 22:23:02	48.30	Completada		FAC-042	2026-07-08 22:23:02	63.25	Completada
	FAC-016	2026-07-08 22:23:02	59.80	Completada		FAC-043	2026-07-08 22:23:02	74.75	Completada
	FAC-017	2026-07-08 22:23:02	34.50	Completada		FAC-044	2026-07-08 22:23:02	66.70	Completada
	FAC-018	2026-07-08 22:23:02	92.00	Completada		FAC-045	2026-07-08 22:23:02	57.50	Completada
	FAC-019	2026-07-08 22:23:02	36.80	Completada		FAC-046	2026-07-08 22:23:02	69.00	Completada
	FAC-020	2026-07-08 22:23:02	63.25	Completada		FAC-047	2026-07-08 22:23:02	97.75	Completada
	FAC-021	2026-07-08 22:23:02	74.75	Completada		FAC-048	2026-07-08 22:23:02	40.25	Completada
	FAC-022	2026-07-08 22:23:02	66.70	Completada		FAC-049	2026-07-08 22:23:02	51.75	Completada
	FAC-023	2026-07-08 22:23:02	57.50	Completada		FAC-050	2026-07-08 22:23:02	63.25	Completada
	FAC-024	2026-07-08 22:23:02	69.00	Completada		FAC-051	2026-07-08 22:23:02	46.00	Completada
	FAC-025	2026-07-08 22:23:02	97.75	Completada		FAC-052	2026-07-08 22:23:02	103.50	Completada
	FAC-026	2026-07-08 22:23:02	40.25	Completada		FAC-053	2026-01-15 10:30:00	134.40	Completada
	FAC-027	2026-07-08 22:23:02	51.75	Completada		FAC-054	2026-02-20 14:15:00	89.60	Completada

7. Registrar Proveedor: Almacena datos completos del proveedor

```

DELIMITER //
CREATE PROCEDURE
registrar_proveedor(
    IN p_ruc
    VARCHAR(15),
    IN
    p_razon_social
    VARCHAR(150),
    IN p_pais
    VARCHAR(50),
    IN p_telefono
    VARCHAR(20),
    IN p_email
    VARCHAR(100)
)
BEGIN
    INSERT INTO
    proveedores
    (ruc_proveedor,
    razon_social,
    pais_origen,
    telefono_contacto,
    email)
    VALUES (p_ruc,
    p_razon_social,
    p_pais, p_telefono,
    p_email);
END //
DELIMITER ;

```

Tabla Original:

	id_proveedor	ruc_proveedor	razon_social	pais_origen	telefono_contacto	email	estado
▶	1	1791234567001	Calzado Ecuatoriano S.A.	Ecuador	022345678	v@calzadoec.com	Activo
	2	0992345678001	Importadora Asiática	China	042987654	i@asiatica.cn	Activo
	3	1893456789001	Cuero Andino Ltda.	Ecuador	032112233	c@cueroandino.ec	Activo
	4	0194567890001	Suelas del Sur	Ecuador	072345678	s@suelas.com	Activo
	5	1795678901001	Global Shoes	Panamá	022667788	i@globalshoes.pa	Activo
	6	0996789012001	Dist. Guayas	Ecuador	042778899	v@distguayas.ec	Activo
	7	1797890123001	Deportes Cia.	Colombia	022889900	d@ymas.co	Activo
	8	0198901234001	Manu Cuenca	Ecuador	072990011	f@mcuenca.ec	Activo
	9	1799012345001	Zapato Premium	Brasil	022112233	p@zapatos.br	Activo
	10	0990123456001	Industrial Ec	Ecuador	042223344	i@calzado.ec	Activo
	11	1191122334001	Loja Shoes	Ecuador	072111222	l@lojashoes.ec	Activo
	12	1792233445001	Textil Foot	Perú	022334455	t@textilfoot.pe	Activo
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Captura de ejecucion Correcta:

```

CALL registrar_proveedor('0999999999001', 'Calzado Ecuatoriano Cia
Ltda', 'Ecuador', '042999999', 'contacto@calzado.ec');

```

 21 20:17:27 SELECT * FROM proveedores LIMIT 0, 1000 12 row(s) returned
 22 20:18:10 CALL registrar_proveedor('0999999999001', 'Calzado Ecuatoriano Cia Ltda', 'Ecuador', '042999999', '...' 1 row(s) affected

Tabla con los cambios del procedimiento:

```

SELECT * FROM proveedores WHERE ruc_proveedor = '0999999999001';

```

	id_proveedor	ruc_proveedor	razon_social	pais_origen	telefono_contacto	email	estado
▶	13	0999999999001	Calzado Ecuatoriano Cia Ltda	Ecuador	042999999	contacto@calzado.ec	Activo
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

8. Actualizar Inventario: Incrementa o disminuye stock según movimiento

```

DELIMITER //
CREATE PROCEDURE
actualizar_inventario(
    IN p_id_producto
    INT,
    IN p_cantidad INT,
    IN
    p_tipo_movimiento
    VARCHAR(30),
    IN p_id_empleado
    INT
)
BEGIN
    IF
    p_tipo_movimiento LIKE
    '%ENTRADA%' THEN
        UPDATE
        productos SET
        stock_actual =
        stock_actual +
        p_cantidad WHERE
        id_producto =
        p_id_producto;
    ELSEIF
    p_tipo_movimiento LIKE
    '%SALIDA%' THEN
        UPDATE
        productos SET
        stock_actual =
        stock_actual -
        p_cantidad WHERE
        id_producto =
        p_id_producto;
    END IF;

    INSERT INTO
    inventario
    (tipo_movimiento,
    cantidad,
    documento_referencia,
    id_producto,
    id_empleado)
    VALUES
    (p_tipo_movimiento,
    p_cantidad, 'Ajuste
    Manual',
    p_id_producto,
    p_id_empleado);
END //
DELIMITER ;

```

Tabla Original:

	id_movimiento	tipo_movimiento	cantidad	fecha_movimiento	documento_referencia	id_producto	id_empleado
▶	1	Entrada	100	2026-07-08 22:23:02	Factura Proveedor 1	1	2
	2	Entrada	80	2026-07-08 22:23:02	Factura Proveedor 2	2	2
	3	Entrada	50	2026-07-08 22:23:02	Factura Proveedor 3	3	2
	4	Entrada	150	2026-07-08 22:23:02	Factura Proveedor 4	4	2
	5	Entrada	60	2026-07-08 22:23:02	Factura Proveedor 5	5	2
	6	Entrada	90	2026-07-08 22:23:02	Factura Proveedor 6	6	2
	7	Entrada	110	2026-07-08 22:23:02	Factura Proveedor 7	7	2
	8	Entrada	45	2026-07-08 22:23:02	Factura Proveedor 8	8	2
	9	Entrada	200	2026-07-08 22:23:02	Factura Proveedor 9	9	2
	10	Entrada	70	2026-07-08 22:23:02	Factura Proveedor 10	10	2
	11	Entrada	40	2026-07-08 22:23:02	Factura Proveedor 11	11	2
	12	Entrada	55	2026-07-08 22:23:02	Factura Proveedor 12	12	2
	13	Entrada	30	2026-07-08 22:23:02	Factura Proveedor 13	13	2
	14	Entrada	130	2026-07-08 22:23:02	Factura Proveedor 14	14	2
	15	Entrada	85	2026-07-08 22:23:02	Factura Proveedor 15	15	2
	16	Entrada	105	2026-07-08 22:23:02	Factura Proveedor 16	16	2
	17	Entrada	140	2026-07-08 22:23:02	Factura Proveedor 17	17	2
	18	Entrada	65	2026-07-08 22:23:02	Factura Proveedor 18	18	2
	19	Entrada	180	2026-07-08 22:23:02	Factura Proveedor 19	19	2
	20	Entrada	50	2026-07-08 22:23:02	Factura Proveedor 20	20	2
	21	Entrada	75	2026-07-08 22:23:02	Factura Proveedor 21	21	2
	22	Entrada	80	2026-07-08 22:23:02	Factura Proveedor 22	22	2
	23	SALIDA VENTA	2	2026-07-12 19:09:50	FAC-999	1	1
	24	SALIDA VENTA	2	2026-07-12 19:16:28	FAC-997	1	1
	25	ENTRADA DEV...	1	2026-07-12 19:24:00	Devolucion Factura 1	1	1
	26	ENTRADA DEV...	1	2026-07-12 19:24:23	Devolucion Factura 1	1	1
	27	ENTRADA AJUSTE	50	2026-07-12 20:21:56	Ajuste Manual	2	1

Captura de ejecucion Correcta:

CALL actualizar_inventario(2, 50, 'ENTRADA AJUSTE', 1);

```

24 20:21:52 CREATE PROCEDURE actualizar_inventario( IN p_id_producto INT, IN p_cantidad INT, IN ... 0 row(s) affected
25 20:21:56 CALL actualizar_inventario(2, 50, 'ENTRADA AJUSTE', 1) 1 row(s) affected

```

Tabla con los cambios del procedimiento:

SELECT * FROM inventario ORDER BY id_movimiento DESC LIMIT 1;

	id_movimiento	tipo_movimiento	cantidad	fecha_movimiento	documento_referencia	id_producto	id_empleado
▶	27	ENTRADA AJUSTE	50	2026-07-12 20:21:56	Ajuste Manual	2	1
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

9. Registrar Queja: Almacena reclamos y sugerencias

```

DELIMITER //
CREATE PROCEDURE
registrar_queja(IN
p_id_cliente INT,
IN p_descripcion
TEXT)
BEGIN
    INSERT INTO
quejas
(id_cliente,
descripcion)
VALUES
(p_id_cliente,
p_descripcion);
END //
DELIMITER ;
CALL
registrar_queja(1,
'El calzado me
quedó grande,
solicito
cambio.');
```

Tabla Original:

	id_queja	id_cliente	descripcion	fecha_queja	estado
▶	1	1	La caja de los zapatos formales llegó completam...	2026-07-12 20:31:36	Pendiente
	2	2	Solicito reembolso, el material del calzado no es ...	2026-07-12 20:31:36	En revisión
	3	3	Los cordones de los botines vinieron rotos y ma...	2026-07-12 20:31:36	Resuelta
	4	4	El calzado Oxford me aprieta demasiado a pesa...	2026-07-12 20:31:36	En revisión
	5	5	Nunca me aplicaron el descuento de la promoció...	2026-07-12 20:31:36	Resuelta
	6	1	El calzado me quedó grande, solicito cambio.	2026-07-12 20:32:40	Pendiente
•	NULL	NULL	NULL	NULL	NULL

Captura de ejecucion Correcta:

CALL registrar_queja(1, 'El calzado me quedó grande, solicito cambio.');

```

2 20:32:36 CREATE PROCEDURE registrar_queja(IN p_id_cliente INT, IN p_descripcion TEXT) BEGIN INSERT... 0 row(s) affected
3 20:32:40 CALL registrar_queja(1, 'El calzado me quedó grande, solicito cambio.') 1 row(s) affected
```

Tabla con los cambios del procedimiento:

SELECT * FROM quejas ORDER BY id_queja DESC LIMIT 1;

	id_queja	id_cliente	descripcion	fecha_queja	estado
▶	6	1	El calzado me quedó grande, solicito cambio.	2026-07-12 20:32:40	Pendiente
•	NULL	NULL	NULL	NULL	NULL

10. Registrar: Auditoría Guarda acción, usuario, fecha y tabla afectada

```
DELIMITER //
CREATE PROCEDURE
registrar_queja(IN
p_id_cliente INT,
IN p_descripcion
TEXT)
BEGIN
    INSERT INTO
quejas
(id_cliente,
descripcion)
VALUES
(p_id_cliente,
p_descripcion);
END //
DELIMITER ;
CALL
registrar_queja(1,
'El calzado me
quedó grande,
solicito
cambio.');
```

```
SELECT * FROM
quejas ORDER BY
id_queja DESC
LIMIT 1;
```

Tabla Original:

	id_auditoria	nombre_tabla	accion_realizada	usuario_db	fecha_hora	detalle_cambios
▶	1	clientes	INSERT	admin_sistema	2026-07-12 20:37:26	Se insertó un nuevo cliente de prueba
*	NULL	NULL	NULL	NULL	NULL	NULL

Captura de ejecucion Correcta:

```
CALL registrar_auditoria('clientes', 'INSERT', 'admin_sistema', 'Se
insertó un nuevo cliente de prueba');
```

```

✓ 6 20:37:17 CREATE PROCEDURE registrar_auditoria( IN p_tabla VARCHAR(50), IN p_accion VARCHAR(...) 0 row(s) affected
✓ 7 20:37:26 CALL registrar_auditoria('clientes', 'INSERT', 'admin_sistema', 'Se insertó un nuevo cliente de prueba') 1 row(s) affected
```

Tabla con los cambios del procedimiento:

```
SELECT * FROM auditoria ORDER BY id_auditoria DESC LIMIT 1;
```

	id_auditoria	nombre_tabla	accion_realizada	usuario_db	fecha_hora	detalle_cambios
▶	1	clientes	INSERT	admin_sistema	2026-07-12 20:37:26	Se insertó un nuevo cliente de prueba
*	NULL	NULL	NULL	NULL	NULL	NULL

5.11 Triggers

1. Auditoría INSERT Clientes Clientes Registra usuario, fecha y operación

```
DELIMITER //
CREATE TRIGGER
trg_audit_insert_clientes
AFTER INSERT ON clientes
FOR EACH ROW
BEGIN
    INSERT INTO auditoria
(nombre_tabla,
accion_realizada,
usuario_db,
detalle_cambios)
VALUES ('clientes',
'INSERT', CURRENT_USER(),
CONCAT('Nuevo cliente
registrado: ',
NEW.identificacion, ' -
',
NEW.nombres_completos));
END //
DELIMITER ;
```

Insertamos un dato de prueba:

```
INSERT INTO clientes (tipo_identificacion, identificacion,
nombres_completos, telefono, email, tipo_cliente)
VALUES ('Cedula', '0000000001', 'Test Trigger Insert', '0999999999',
'test@mail.com', 'Minorista');
```

6 19:00:30 INSERT INTO clientes (tipo_identificacion, identificacion, nombres_completos, telefono, email, tipo_clien...

1 row(s) affected

Verificacion:

```
SELECT * FROM auditoria
WHERE accion_realizada = 'INSERT' AND nombre_tabla = 'clientes'
ORDER BY id_auditoria DESC LIMIT 1;
```

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

Fetch rows:

	id_auditoria	nombre_tabla	accion_realizada	usuario_db	fecha_hora	detalle_cambios
▶	2	clientes	INSERT	root@localhost	2026-07-13 19:00:30	Nuevo cliente registrado: 0000000001 - Test Trigger Insert
✱	NULL	NULL	NULL	NULL	NULL	NULL

2. Auditoría UPDATE Clientes Clientes Registra valores anteriores y nuevos

```
DELIMITER //
CREATE TRIGGER
trg_audit_update_clientes
AFTER UPDATE ON clientes
FOR EACH ROW
BEGIN
    INSERT INTO auditoria
(nombre_tabla,
accion_realizada,
usuario_db,
detalle_cambios)
VALUES ('clientes',
'UPDATE', CURRENT_USER(),
CONCAT('ID: ',
OLD.identificacion, ' |
Antes: ',
OLD.nombres_completos, '
| Después: ',
NEW.nombres_completos));
END //
DELIMITER ;
```

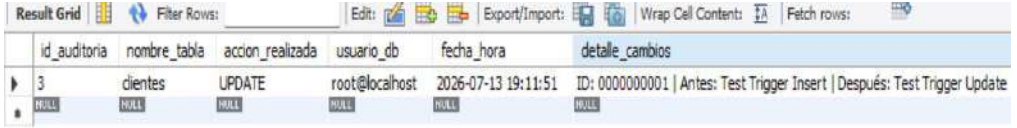
Actualizamos un dato de prueba (en este caso el dato del punto 1):

```
UPDATE clientes SET nombres_completos = 'Test Trigger Update' WHERE
identificacion = '0000000001';
```

8 19:11:44 CREATE TRIGGER trg_audit_update_clientes AFTER UPDATE ON clientes FOR EACH ROW BEGI... 0 row(s) affected
9 19:11:51 UPDATE clientes SET nombres_completos = 'Test Trigger Update' WHERE identificacion = '000000... 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

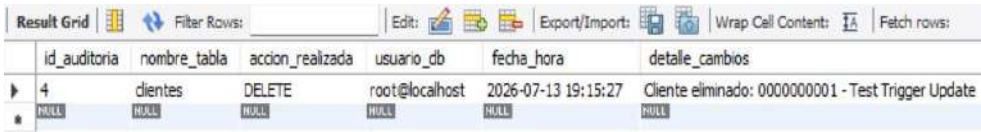
Verificacion:

```
SELECT * FROM auditoria
WHERE accion_realizada = 'INSERT' AND nombre_tabla = 'clientes'
ORDER BY id_auditoria DESC LIMIT 1;
```

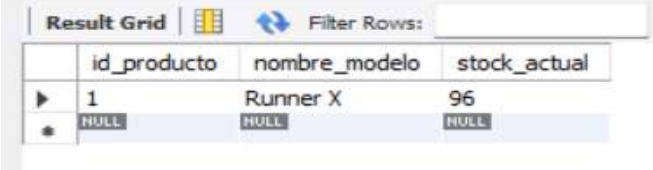


id_auditoria	nombre_tabla	accion_realizada	usuario_db	fecha_hora	detalle_cambios
3	clientes	UPDATE	root@localhost	2026-07-13 19:11:51	ID: 0000000001 Antes: Test Trigger Insert Después: Test Trigger Update
NULL	NULL	NULL	NULL	NULL	NULL

3. Auditoría DELETE Clientes Clientes Registra el registro eliminado

<pre> DELIMITER // CREATE TRIGGER trg_audit_delete_clientes AFTER DELETE ON clientes FOR EACH ROW BEGIN INSERT INTO auditoria (nombre_tabla, accion_realizada, usuario_db, detalle_cambios) VALUES ('clientes', 'DELETE', CURRENT_USER(), CONCAT('Cliente eliminado: ', OLD.identificacion, ' - ', OLD.nombres_completos)); END // DELIMITER ; </pre>	Borramos un dato de prueba: <pre>DELETE FROM clientes WHERE identificacion = '0000000001';</pre> 11 19:15:13 CREATE TRIGGERtrg_audit_delete_clientes AFTER DELETE ON clientes FOR EACH ROW BEGL... 0 row(s) affected 12 19:15:27 DELETE FROM clientes WHERE identificacion = '0000000001' 1 row(s) affected Verificacion: <pre> SELECT * FROM auditoria WHERE accion_realizada = 'DELETE' AND nombre_tabla = 'clientes' ORDER BY id_auditoria DESC LIMIT 1; </pre> 
---	---

4. Descontar Stock por Venta DetalleVenta Reduce inventario automáticamente al insertar

<pre> DELIMITER // CREATE TRIGGER trg_descontar_stock_venta AFTER INSERT ON detalle_venta FOR EACH ROW BEGIN UPDATE productos SET stock_actual = stock_actual - NEW.cantidad WHERE id_producto = NEW.id_producto; END // DELIMITER ; </pre>	Insertamos un detalle simulando que se vendieron 2 pares del producto 1: <pre>INSERT INTO detalle_venta (cantidad, precio_unitario, subtotal_linea, id_venta, id_producto) VALUES (2, 50.00, 100.00, 1, 1);</pre> 14 19:18:41 CREATE TRIGGERtrg_descontar_stock_venta AFTER INSERT ON detalle_venta FOR EACH RO... 0 row(s) affected 15 19:21:38 INSERT INTO detalle_venta (cantidad, precio_unitario, subtotal_linea, id_venta, id_producto) VALU... 1 row(s) affected Verificacion:  El stock de ese producto debe haberse restado <pre> SELECT * FROM auditoria WHERE accion_realizada = 'INSERT' AND nombre_tabla = 'clientes' ORDER BY id_auditoria DESC LIMIT 1; </pre> 
---	---

5. Incrementar Stock por Devolución Devoluciones Incrementa inventario al registrar devolución

```
DELIMITER //
CREATE TRIGGER
trg_incrementar_stock_devolucion
AFTER INSERT ON devoluciones
FOR EACH ROW
BEGIN
    UPDATE productos
    SET stock_actual =
stock_actual +
NEW.cantidad_devuelta
WHERE id_producto =
NEW.id_producto;
END //
DELIMITER ;
```

Devolvemos dos pares para la ejecucion del trigger:

```
INSERT INTO devoluciones (motivo, cantidad_devuelta, estado_calzado,
id_venta, id_producto, id_empleado)
VALUES ('Prueba de Trigger', 2, 'Nuevo', 1, 1, 1);
```

19 19:31:22 CREATE TRIGGER trg_incrementar_stock_devolucion AFTER INSERT ON devoluciones FOR EAC... 0 row(s) affected
20 19:31:27 INSERT INTO devoluciones (motivo, cantidad_devuelta, estado_calzado, id_venta, id_producto, id_... 1 row(s) affected

Verificacion:

Result Grid			
Filter Rows:			
	id_producto	nombre_modelo	stock_actual
▶	1	Runner X	94
•	NULL	NULL	NULL

El stock de ese producto debe haber sumado 2

```
SELECT id_producto, nombre_modelo, stock_actual
FROM productos
WHERE id_producto = 1;
```

	id_producto	nombre_modelo	stock_actual
▶	1	Runner X	96
•	NULL	NULL	NULL

6. Control de Stock Negativo Inventario/DetalleVenta Impide que el stock sea menor a cero

```
DELIMITER //
CREATE TRIGGER
trg_control_stock_negativo
BEFORE UPDATE ON productos
FOR EACH ROW
BEGIN
    IF NEW.stock_actual <
0 THEN
        SIGNAL SQLSTATE
'45000' SET MESSAGE_TEXT =
'Error: El stock no puede
ser menor a cero.';
    END IF;
END //
DELIMITER ;
```

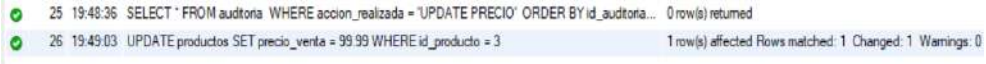
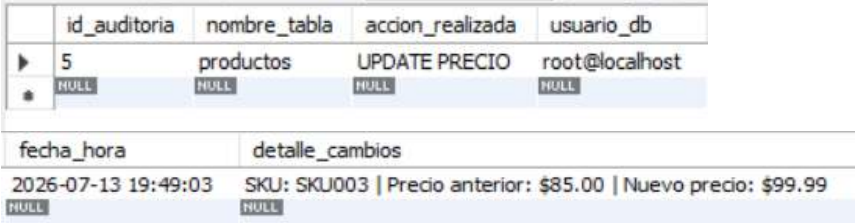
Verificacion Intentamos poner un stock negativo:

```
UPDATE productos SET stock_actual = -5 WHERE id_producto = 2;
```

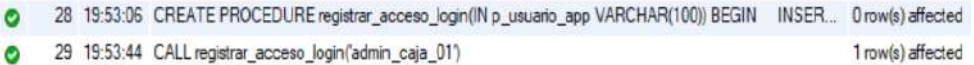
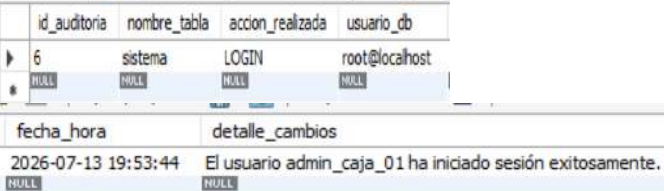
23 19:44:43 UPDATE productos SET stock_actual = -5 WHERE id_producto = 2 Error Code: 1644. Error: El stock no puede ser menor a cero.

19:44:43 UPDATE productos SET stock_actual = -5 WHERE id_producto = 2 Error Code: 1644. Error: El stock no puede ser menor a cero. 0.016 sec

7. Registro de Cambio de Precio Productos Guarda precio anterior y precio nuevo

<pre> DELIMITER // CREATE TRIGGER trg_cambioPrecio AFTER UPDATE ON productos FOR EACH ROW BEGIN IF OLD.precio_venta != NEW.precio_venta THEN INSERT INTO auditoria (nombre_tabla, accion_realizada, usuario_db, detalle_cambios) VALUES ('productos', 'UPDATE PRECIO', CURRENT_USER(), CONCAT('SKU: ', NEW.sku, ' Precio anterior: \$', OLD.precio_venta, ' Nuevo precio: \$', NEW.precio_venta)); END IF; END // DELIMITER ; </pre>	Cambiamos el precio del producto 3: UPDATE productos SET precio_venta = 99.99 WHERE id_producto = 3;  Verificacion: SELECT * FROM auditoria WHERE accion_realizada = 'UPDATE PRECIO' ORDER BY id_auditoria DESC LIMIT 1; 
--	---

8. Registro de Acceso Auditoría Guarda usuario, fecha y hora de acceso

<pre> DELIMITER // CREATE PROCEDURE registrar_acceso_login(IN p_usuario_app VARCHAR(100)) BEGIN INSERT INTO auditoria (nombre_tabla, accion_realizada, usuario_db, detalle_cambios) VALUES ('sistema', 'LOGIN', CURRENT_USER(), CONCAT('El usuario ', p_usuario_app, ' ha iniciado sesión exitosamente.')); END // DELIMITER ; </pre>	Simulamos un login desde el sistema: CALL registrar_acceso_login('admin_caja_01');  Verificacion: SELECT * FROM auditoria WHERE accion_realizada = 'LOGIN' ORDER BY id_auditoria DESC LIMIT 1; 
---	---

5.12 Transacciones

1. Venta Completa ventas, detalle_venta, inventario, promociones COMMIT si correcto / ROLLBACK si error

```
DELIMITER //
CREATE PROCEDURE tx_venta(IN p_fac VARCHAR(20), IN p_cli INT, IN p_emp INT, IN
p_prod INT, IN p_cant INT, IN p_pago VARCHAR(20))
BEGIN
    DECLARE v_precio, v_sub, v_iva, v_tot DECIMAL(10,2);
    DECLARE v_stock, v_id INT;

    -- Manejador de errores
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Error: Se aplicó ROLLBACK' AS Estado;
    END;
    START TRANSACTION;
    SELECT precio_venta, stock_actual INTO v_precio, v_stock
    FROM productos WHERE id_producto = p_prod FOR UPDATE;

    IF v_stock >= p_cant THEN
        SET v_sub = v_precio * p_cant, v_iva = v_sub * 0.15, v_tot = v_sub +
v_iva;

        INSERT INTO ventas (numero_factura, subtotal, iva, total_factura,
metodo_pago, id_cliente, id_empleado)
        VALUES (p_fac, v_sub, v_iva, v_tot, p_pago, p_cli, p_emp);
        SET v_id = LAST_INSERT_ID();

        INSERT INTO detalle_venta (cantidad, precio_unitario, subtotal_linea,
id_venta, id_producto)
        VALUES (p_cant, v_precio, v_sub, v_id, p_prod);

        COMMIT;
        SELECT 'Éxito: Se guardó la venta (COMMIT)' AS Estado;
    ELSE
        ROLLBACK;
        SELECT 'Error: Sin stock (ROLLBACK)' AS Estado;
    END IF;
END //
DELIMITER ;
```

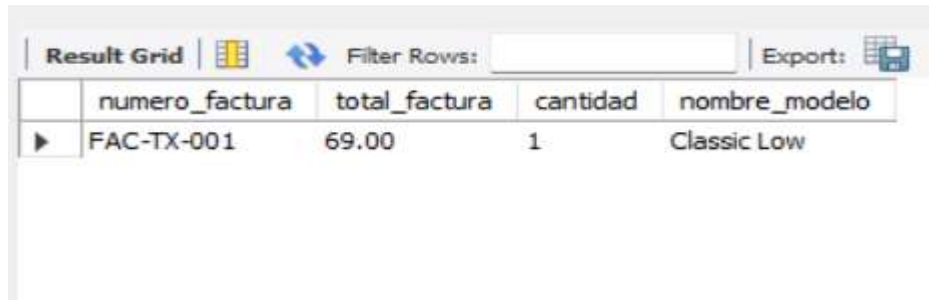
EJECUCIÓN:

CALL tx_venta('FAC-TX-001', 1, 1, 2, 1, 'Tarjeta');

```
32 20:22:13 CALL tx_venta('FAC-TX-001', 1, 1, 2, 1, 'Tarjeta') 1 row(s) returned
33 20:24:09 SELECT v.numero_factura, v.total_factura, d.cantidad, p.nombre_modelo FROM ventas v JOIN detal... 1 row(s) returned
```

VERIFICACIÓN:

```
SELECT v.numero_factura, v.total_factura, d.cantidad,
p.nombre_modelo
FROM ventas v
JOIN detalle_venta d ON v.id_venta = d.id_venta
JOIN productos p ON d.id_producto = p.id_producto
WHERE v.numero_factura = 'FAC-TX-001';
```



	numero_factura	total_factura	cantidad	nombre_modelo
▶	FAC-TX-001	69.00	1	Classic Low

2. Devolución Completa devoluciones, inventario Atomicidad y consistencia

```
DELIMITER //
CREATE PROCEDURE tx_precios(IN p_cat INT, IN p_porc
DECIMAL(5,2))
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Error: ROLLBACK ejecutado' AS Estado;
    END;

    START TRANSACTION;

    UPDATE productos
    SET precio_venta = precio_venta * (1 + (p_porc / 100))
    WHERE id_categoria = p_cat;

    INSERT INTO auditoria (nombre_tabla, accion_realizada,
usuario_db, detalle_cambios)
    VALUES ('productos', 'UPDATE MASIVO', CURRENT_USER(),
CONCAT('Aumento ', p_porc, '% a cat ', p_cat));

    COMMIT;
    SELECT 'Éxito: Cambios guardados (COMMIT)' AS Estado;
END //
DELIMITER ;
```

EJECUCIÓN:

CALL tx_precios(1, 10.00);

35	20:40:43	CREATE PROCEDURE tx_precios(IN p_cat INT, IN p_porc DECIMAL(5,2)) BEGIN DECLARE EXI...	0 row(s) affected
36	20:40:47	CALL tx_precios(1, 10.00)	1 row(s) returned

VERIFICACIÓN:

SELECT id_producto, nombre_modelo, precio_venta
FROM productos WHERE id_categoria = 1;

Result Grid		 Filter Rows:	 Edit:
	id_producto	nombre_modelo	precio_venta
▶	1	Runner X	55.00
	6	Runner Pro	60.50
	11	Trail Max	82.50
	16	Marathon Lite	57.20
	21	Soccer Cleat	71.50
	25	Oxford Ejecutivo (Nacional)	71.50
	NULL	NULL	NULL

3. Compra a Proveedor compras, detalle_compra, inventario Integridad en registro y stock

```

DELIMITER //
CREATE PROCEDURE
tx_compra_proveedor(IN p_prod INT, IN
p_cant INT, IN p_emp INT, IN p_doc
VARCHAR(100))
BEGIN
    DECLARE EXIT HANDLER FOR
    SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Error: Falla de
Integridad. ROLLBACK aplicado.' AS
Estado;
    END;

    START TRANSACTION;

    UPDATE productos
    SET stock_actual = stock_actual +
p_cant
    WHERE id_producto = p_prod;

    INSERT INTO inventario
(tipo_movimiento, cantidad,
documento_referencia, id_producto,
id_empleado)
VALUES ('ENTRADA COMPRA', p_cant,
p_doc, p_prod, p_emp);

    COMMIT;
    SELECT 'Éxito: Compra registrada
con integridad (COMMIT)' AS Estado;
END //
DELIMITER ;

```

EJECUCIÓN:

Ingresan 100 unidades del producto 1 con la Factura PROV-123

CALL tx_compra_proveedor(1, 100, 2, 'Factura PROV-123');

38	20:46:47	CREATE PROCEDURE tx_compra_proveedor(IN p_prod INT, IN p_cant INT, IN p_emp INT, IN p_d...	0 row(s) affected
39	20:46:47	-- EJECUCIÓN CALL tx_compra_proveedor(1, 100, 2, 'Factura PROV-123')	1 row(s) returned

Estado
▶ Éxito: Compra registrada con integridad (COMMIT)

VERIFICACIÓN:

```

SELECT p.id_producto, p.nombre_modelo, p.stock_actual, i.tipo_movimiento,
i.documento_referencia
FROM productos p JOIN inventario i ON p.id_producto = i.id_producto
WHERE i.documento_referencia = 'Factura PROV-123';

```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	id_producto	nombre_modelo	stock_actual	tipo_movimiento	documento_referencia
▶	1	Runner X	196	ENTRADA COMPRA	Factura PROV-123

4. Actualización Masiva de Inventario inventario Demuestra ACID: Atomicidad, Consistencia, Aislamiento, Durabilidad

```

DELIMITER //
CREATE PROCEDURE tx_masiva_inventario(IN
p_prov INT, IN p_bono_stock INT)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Error: Se rompió la
Atomicidad. ROLLBACK ejecutado.' AS Estado;
    END;

    START TRANSACTION;

    -- AISLAMIENTO Y CONSISTENCIA: Bloquea
registros y suma el stock a todo un
proveedor
    UPDATE productos
    SET stock_actual = stock_actual +
p_bono_stock
    WHERE id_proveedor = p_prov;

    -- Rastreo de auditoría masivo
    INSERT INTO auditoria (nombre_tabla,
accion_realizada, usuario_db,
detalle_cambios)
VALUES ('productos', 'AJUSTE MASIVO
ACID', CURRENT_USER(), CONCAT('Bono de ',
p_bono_stock, ' unidades al proveedor ',
p_prov));

    -- DURABILIDAD: Guarda permanentemente
COMMIT;
    SELECT 'Éxito: Actualización masiva ACID
completada (COMMIT)' AS Estado;
END //
DELIMITER ;

```

EJECUCIÓN:

Le sumamos 20 unidades a todos los productos del proveedor 1

CALL tx_masiva_inventario(1, 20);

Estado
Éxito: Actualización masiva ACID completada (COMMIT)

✓	41	20:54:12	CREATE PROCEDURE tx_masiva_inventario(IN p_prov INT, IN p_bono_stock INT) BEGIN DECL...	0 row(s) affected
✓	42	20:54:16	CALL tx_masiva_inventario(1, 20)	1 row(s) returned

VERIFICACIÓN:

```

SELECT nombre_tabla, accion_realizada, detalle_cambios
FROM auditoria WHERE accion_realizada = 'AJUSTE MASIVO ACID'
ORDER BY id_auditoria DESC LIMIT 1;

```

nombre_tabla	accion_realizada	detalle_cambios
productos	AJUSTE MASIVO ACID	Bono de 20 unidades al proveedor 1

5.13 Usuarios, Roles y Privilegios

1. Administrador usuario_admin SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, DROP (toda la BD)

Rol: Administrador

```
CREATE USER 'usuario_admin'@'localhost' IDENTIFIED BY 'admin123';
```

1 21:19:18 CREATE USER 'usuario_admin'@'localhost' IDENTIFIED BY 'admin123' 0 row(s) affected

Asignación de control total sobre la base de datos:

```
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, DROP  
ON SistemaCalzado_2026.* TO 'usuario_admin'@'localhost';  
FLUSH PRIVILEGES;
```

2 21:19:18 GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, DROP ON SistemaCalzado_2026.* TO 'usuario_admin'@'localhost' 0 row(s) affected

3 21:19:18 FLUSH PRIVILEGES 0 row(s) affected

Verificación:

```
SHOW GRANTS FOR 'usuario_admin'@'localhost';
```

	Grants for usuario_admin@localhost
▶	GRANT USAGE ON *.* TO 'usuario_admin'@'localhost'
	GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, ALTER ON 'sistemacalzado_2026'.* TO 'usuario_admin'@'localhost'

2. Gerente usuario_gerente SELECT en ventas, clientes, productos, vistas. EXECUTE en procedimientos de reportes

Rol: Gerente

```
CREATE USER 'usuario_gerente'@'localhost' IDENTIFIED BY 'gerente123';
```

Action Output

#	Time	Action	Message
1	21:30:19	CREATE USER 'usuario_gerente'@'localhost' IDENTIFIED BY 'gerente123'	0 row(s) affected

Asignación de control sobre la base de datos

Permisos de lectura en tablas clave y vistas:

```
GRANT SELECT ON SistemaCalzado_2026.ventas TO 'usuario_gerente'@'localhost';  
GRANT SELECT ON SistemaCalzado_2026.clientes TO 'usuario_gerente'@'localhost';  
GRANT SELECT ON SistemaCalzado_2026.productos TO 'usuario_gerente'@'localhost';  
GRANT SELECT ON SistemaCalzado_2026.VentasConsolidadas TO 'usuario_gerente'@'localhost';  
GRANT SELECT ON SistemaCalzado_2026.DesempenoVendedores TO 'usuario_gerente'@'localhost';
```

2 21:31:21 GRANT SELECT ON SistemaCalzado_2026.ventas TO 'usuario_gerente'@'localhost' 0 row(s) affected

3 21:31:21 GRANT SELECT ON SistemaCalzado_2026.clientes TO 'usuario_gerente'@'localhost' 0 row(s) affected

4 21:31:21 GRANT SELECT ON SistemaCalzado_2026.productos TO 'usuario_gerente'@'localhost' 0 row(s) affected

5 21:31:21 GRANT SELECT ON SistemaCalzado_2026.VentasConsolidadas TO 'usuario_gerente'@'localhost' 0 row(s) affected

6 21:31:21 GRANT SELECT ON SistemaCalzado_2026.DesempenoVendedores TO 'usuario_gerente'@'localhost' 0 row(s) affected

Permisos para ejecutar procedimientos de reportes:

```
GRANT EXECUTE ON PROCEDURE SistemaCalzado_2026.calcular_ventas_mensuales TO 'usuario_gerente'@'localhost';
```

```
GRANT EXECUTE ON PROCEDURE SistemaCalzado_2026.clientes_frecuentes TO 'usuario_gerente'@'localhost';  
FLUSH PRIVILEGES;
```

✓	13	21:35:45	GRANT EXECUTE ON PROCEDURE SistemaCalzado_2026.calcular_ventas_mensuales TO 'usuario_gerente'@'localhost';	0 row(s) affected
✓	14	21:35:45	GRANT EXECUTE ON PROCEDURE SistemaCalzado_2026.clientes_frecuentes TO 'usuario_gerente'@'localhost';	0 row(s) affected
✓	15	21:35:45	FLUSH PRIVILEGES	0 row(s) affected

Verificacion:

```
SHOW GRANTS FOR 'usuario_gerente'@'localhost';
```

Grants for usuario_gerente@localhost	
▶	GRANT USAGE ON *.* TO 'usuario_gerente'@'localhost'
	GRANT SELECT ON `sistemacalzado_2026`.`clientes` TO 'usuario_gerente'@'localhost'
	GRANT SELECT ON `sistemacalzado_2026`.`desempenovendedores` TO 'usuario_gerente'@'localhost'
	GRANT SELECT ON `sistemacalzado_2026`.`productos` TO 'usuario_gerente'@'localhost'
	GRANT SELECT ON `sistemacalzado_2026`.`ventas` TO 'usuario_gerente'@'localhost'
	GRANT SELECT ON `sistemacalzado_2026`.`ventasconsolidadas` TO 'usuario_gerente'@'localhost'
	GRANT EXECUTE ON PROCEDURE `sistemacalzado_2026`.`calcular_ventas_mensuales` TO 'usuario_gerente'@'localhost'
	GRANT EXECUTE ON PROCEDURE `sistemacalzado_2026`.`clientes_frecuentes` TO 'usuario_gerente'@'localhost'

3. Cajero usuario_cajero SELECT clientes/productos. INSERT ventas/detalle_venta. EXECUTE registrar_venta

Rol: Cajero

```
CREATE USER 'usuario_cajero'@'localhost' IDENTIFIED BY 'cajero123';
```

✓	17	21:38:22	CREATE USER 'usuario_cajero'@'localhost' IDENTIFIED BY 'cajero123';	0 row(s) affected
---	----	----------	---	-------------------

Permisos de lectura en catálogos y escritura en ventas

```
GRANT SELECT ON SistemaCalzado_2026.clientes TO 'usuario_cajero'@'localhost';
```

```
GRANT SELECT ON SistemaCalzado_2026.productos TO 'usuario_cajero'@'localhost';
```

```
GRANT INSERT ON SistemaCalzado_2026.ventas TO 'usuario_cajero'@'localhost';
```

```
GRANT INSERT ON SistemaCalzado_2026.detalle_venta TO 'usuario_cajero'@'localhost';
```

✓	18	21:39:13	GRANT SELECT ON SistemaCalzado_2026.clientes TO 'usuario_cajero'@'localhost';	0 row(s) affected
✓	19	21:39:13	GRANT SELECT ON SistemaCalzado_2026.productos TO 'usuario_cajero'@'localhost';	0 row(s) affected
✓	20	21:39:13	GRANT INSERT ON SistemaCalzado_2026.ventas TO 'usuario_cajero'@'localhost';	0 row(s) affected

Permiso exclusivo para ejecutar la transacción de venta

```
GRANT EXECUTE ON PROCEDURE SistemaCalzado_2026.registrar_venta TO 'usuario_cajero'@'localhost';
```

✓	21	21:39:13	GRANT INSERT ON SistemaCalzado_2026.detalle_venta TO 'usuario_cajero'@'localhost';	0 row(s) affected
---	----	----------	--	-------------------

✓	22	21:40:24	FLUSH PRIVILEGES	0 row(s) affected
---	----	----------	------------------	-------------------

Verificacion:

```
SHOW GRANTS FOR 'usuario_cajero'@'localhost';
```

Grants for usuario_cajero@localhost	
▶	GRANT USAGE ON *.* TO 'usuario_cajero'@'localhost'
	GRANT SELECT ON `sistemacalzado_2026`.`clientes` TO 'usuario_cajero'@'localhost'
	GRANT INSERT ON `sistemacalzado_2026`.`detalle_venta` TO 'usuario_cajero'@'localhost'
	GRANT SELECT ON `sistemacalzado_2026`.`productos` TO 'usuario_cajero'@'localhost'
	GRANT INSERT ON `sistemacalzado_2026`.`ventas` TO 'usuario_cajero'@'localhost'

4. Vendedor usuario_vendedor SELECT clientes/productos/vistas. INSERT ventas propias

Rol: Vendedor

```
CREATE USER 'usuario_vendedor'@'localhost' IDENTIFIED BY 'vendedor123';
```

24 21:47:30 CREATE USER 'usuario_vendedor'@'localhost' IDENTIFIED BY 'vendedor123'

Permisos de lectura y registro de sus ventas:

```
GRANT SELECT ON SistemaCalzado_2026.clientes TO 'usuario_vendedor'@'localhost';
GRANT SELECT ON SistemaCalzado_2026.productos TO 'usuario_vendedor'@'localhost';
GRANT SELECT ON SistemaCalzado_2026.VentasConsolidadas TO 'usuario_vendedor'@'localhost';
GRANT INSERT ON SistemaCalzado_2026.ventas TO 'usuario_vendedor'@'localhost';
GRANT INSERT ON SistemaCalzado_2026.detalle_venta TO 'usuario_vendedor'@'localhost';
FLUSH PRIVILEGES;
```

25	21:47:30	GRANT SELECT ON SistemaCalzado_2026.clientes TO 'usuario_vendedor'@'localhost'	0 row(s) affected
26	21:47:30	GRANT SELECT ON SistemaCalzado_2026.productos TO 'usuario_vendedor'@'localhost'	0 row(s) affected
27	21:47:30	GRANT SELECT ON SistemaCalzado_2026.VentasConsolidadas TO 'usuario_vendedor'@'localhost'	0 row(s) affected
28	21:47:30	GRANT INSERT ON SistemaCalzado_2026.ventas TO 'usuario_vendedor'@'localhost'	0 row(s) affected
29	21:47:30	GRANT INSERT ON SistemaCalzado_2026.detalle_venta TO 'usuario_vendedor'@'localhost'	0 row(s) affected
30	21:47:30	FLUSH PRIVILEGES	0 row(s) affected

Verificacion:

```
SHOW GRANTS FOR 'usuario_vendedor'@'localhost';
```

Grants for usuario_vendedor@localhost	
▶	GRANT USAGE ON *.* TO 'usuario_vendedor'@'localhost'
	GRANT SELECT ON `sistemacalzado_2026`.`clientes` TO 'usuario_vendedor'@'localhost'
	GRANT INSERT ON `sistemacalzado_2026`.`detalle_venta` TO 'usuario_vendedor'@'localhost'
	GRANT SELECT ON `sistemacalzado_2026`.`productos` TO 'usuario_vendedor'@'localhost'
	GRANT INSERT ON `sistemacalzado_2026`.`ventas` TO 'usuario_vendedor'@'localhost'
	GRANT SELECT ON `sistemacalzado_2026`.`ventasconsolidadas` TO 'usuario_vendedor'@'localhost'

5. Auditor usuario_auditor SELECT en auditoría, vistas de auditoría y reportes. Sin INSERT/UPDATE/DELETE

Rol: Administrador

```
CREATE USER 'usuario_auditor'@'localhost' IDENTIFIED BY 'auditor123';
```

32 21:52:15 CREATE USER 'usuario_auditor'@'localhost' IDENTIFIED BY 'auditor123' 0 row(s) affected

Acceso exclusivo de lectura a registros de control:

```
GRANT SELECT ON SistemaCalzado_2026.auditoria TO 'usuario_auditor'@'localhost';
GRANT SELECT ON SistemaCalzado_2026.DevolucionesDetalladas TO 'usuario_auditor'@'localhost';
GRANT SELECT ON SistemaCalzado_2026.ProductosBajoStock TO 'usuario_auditor'@'localhost';
FLUSH PRIVILEGES;
```

32	21:52:15	CREATE USER 'usuario_auditor'@'localhost' IDENTIFIED BY 'auditor123'	0 row(s) affected
33	21:52:15	GRANT SELECT ON SistemaCalzado_2026.auditoria TO 'usuario_auditor'@'localhost'	0 row(s) affected
34	21:52:15	GRANT SELECT ON SistemaCalzado_2026.DevolucionesDetalladas TO 'usuario_auditor'@'localhost'	0 row(s) affected
35	21:52:16	GRANT SELECT ON SistemaCalzado_2026.ProductosBajoStock TO 'usuario_auditor'@'localhost'	0 row(s) affected
36	21:52:16	FLUSH PRIVILEGES	0 row(s) affected

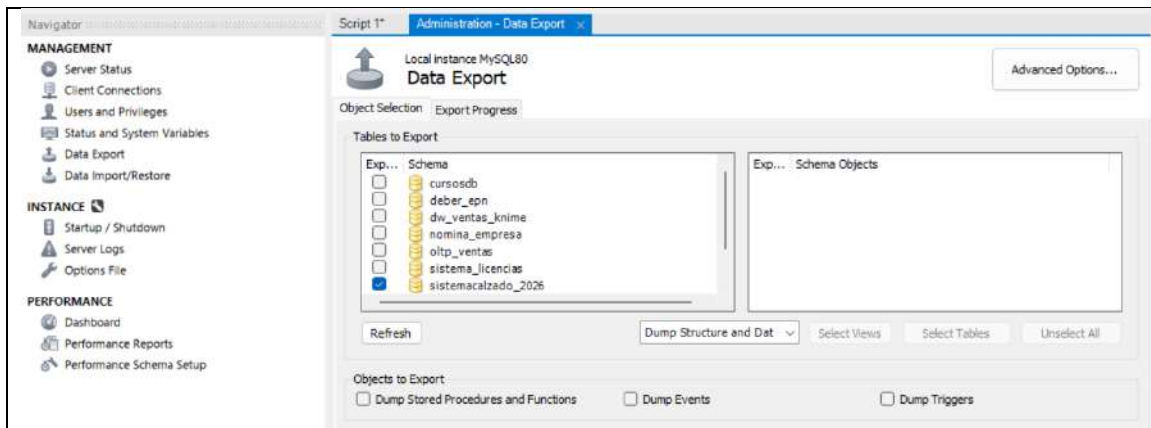
Verificacion:

```
SHOW GRANTS FOR 'usuario_admin'@'localhost';
```

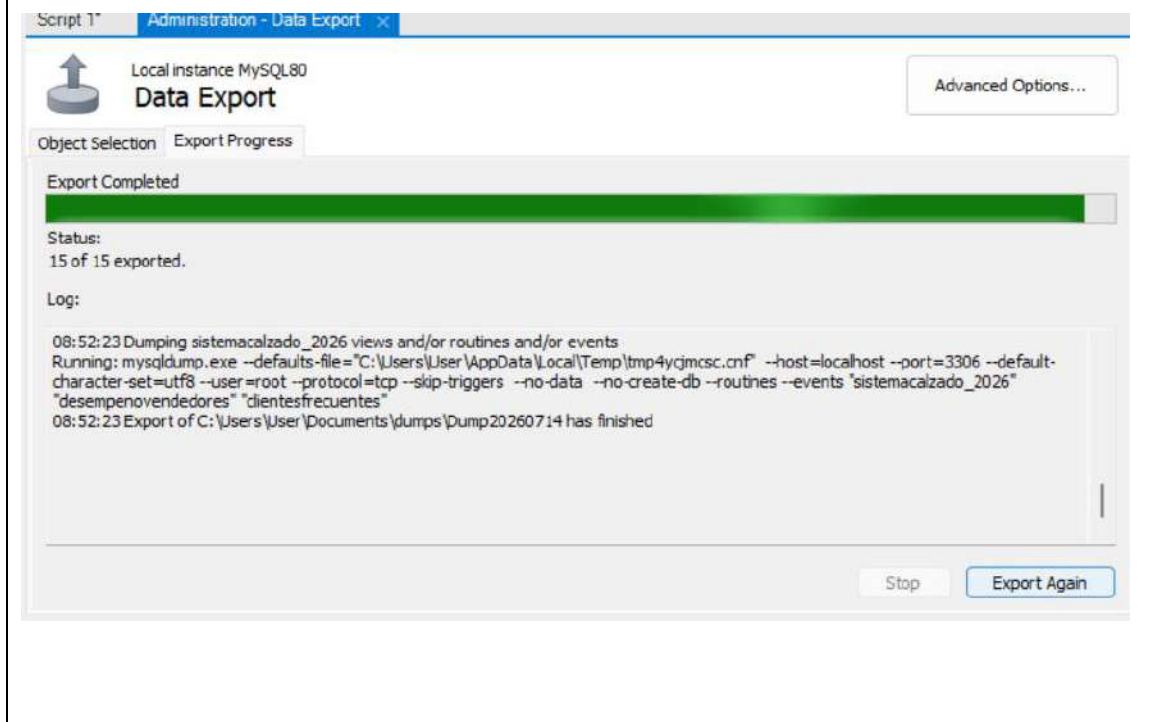
Grants for usuario_auditor@localhost	
▶	GRANT USAGE ON *.* TO 'usuario_auditor'@'localhost'
	GRANT SELECT ON `sistemacalzado_2026`.`auditoria` TO 'usuario_auditor'@'localhost'
	GRANT SELECT ON `sistemacalzado_2026`.`devolucionesdetalladas` TO 'usuario_auditor'@'localhost'
	GRANT SELECT ON `sistemacalzado_2026`.`productosbajostock` TO 'usuario_auditor'@'localhost'

5.14 Respaldo y Recuperación

CREACIÓN DE LA TABLA DE BITÁCORA DE RESPALDOS: <pre>CREATE TABLE registro_respaldos (id_respaldo INT AUTO_INCREMENT PRIMARY KEY, tipo_respaldo VARCHAR(50) NOT NULL, nombre_backup VARCHAR(100) NOT NULL, fecha_backup DATE NOT NULL, hora_backup TIME NOT NULL, usuario_responsable VARCHAR(100) NOT NULL);</pre> La tabla registro_respaldos no se incluyó en los diagramas iniciales (MER) porque no pertenece al Core Business de la empresa (ventas, clientes), sino a la administración del servidor (DBA). Aplicando el principio de separación de responsabilidades, las estructuras de contingencia se definen en la fase de despliegue, manteniendo separada la lógica comercial de la operativa.	REGISTRO DE LOS RESPALDOS SOLICITADOS: <pre>INSERT INTO registro_respaldos (tipo_respaldo, nombre_backup, fecha_backup, hora_backup, usuario_responsable) VALUES ('Respaldo inicial', 'backup_inicial_v1.sql', CURDATE(), CURTIME(), CURRENT_USER()), ('Respaldo intermedio', 'backup_intermedio_v2.sql', CURDATE(), CURTIME(), CURRENT_USER()), ('Respaldo final', 'backup_final_v3.sql', CURDATE(), CURTIME(), CURRENT_USER());</pre>
COMANDOS REALES EN CONSOLA DEL SISTEMA (CMD): Ojo aquí se debe poner el comando con la ruta completa en la que se encuentra la base de datos Ejemplo: & "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqldump.exe" -u root -p SistemaCalzado_2026 > & "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqldump.exe" -u root -p SistemaCalzado_2026 > C:\Users\Public\backup_inicial_v1.sql mysqldump -u root -p SistemaCalzado_2026 > backup_inicial_v1.sql mysqldump -u root -p SistemaCalzado_2026 > backup_intermedio_v2.sql mysqldump -u root -p SistemaCalzado_2026 > backup_final_v3.sql EXPORTACION DESDE MySQL Workbench 1. Abrir el MySQL Workbench. 2. En el menú inferior, haz clic en Administración y luego elige Data Export. 3. En la lista de schemas, marca la casilla de la base de datos: SistemaCalzado_2026.	



4. Haz clic en el botón **Start Export** (la pestaña a la derecha de object selection).



6. PRESENTACIÓN

El proyecto consolida la estructura y documentación técnica en este informe PDF. Además, se adjuntarán los scripts SQL pertinentes y el trabajo será respaldado en la plataforma GitHub para el control de versiones. Todo el equipo se encuentra preparado para la sustentación teórica y técnica del Modelo Entidad-Relación y los demás elementos implementados.

7. RECURSOS NECESARIOS

Para replicar o auditar el entorno, se requiere:

- **Motor de Base de Datos y Gestor:** MySQL Workbench / phpMyAdmin.
- **Diseño Conceptual y Lógico:** Mermaid.live o MySQL Workbench.
- **Editor de Código:** Visual Studio Code (VS Code) y MySQL Workbench.
- **Control de Versiones:** Git y Repositorio en GitHub.

8. CONCLUSIONES

- Se diseñó e implementó exitosamente una base de datos relacional normalizada hasta la Tercera Forma Normal (3FN), garantizando la integridad referencial y eliminando por completo las redundancias de datos presentes en los registros históricos y sistemas heredados de la empresa.
- La automatización de las reglas de negocio críticas mediante el uso de Triggers (disparadores) y Procedimientos Almacenados demostró ser una solución altamente efectiva para mantener la consistencia del inventario en tiempo real, previniendo errores humanos durante las transacciones de ventas y devoluciones.
- La estructuración de bloques transaccionales controlados garantiza el cumplimiento de las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad). Esto asegura que operaciones financieras y de stock complejas se ejecuten de forma segura, protegiendo los datos corporativos ante cualquier fallo parcial del sistema.
- El esquema de seguridad implementado a través de un Control de Acceso Basado en Roles (RBAC) cumple rigurosamente con los requerimientos operativos de la nueva administración, limitando la exposición de datos sensibles y manteniendo una trazabilidad exacta mediante el registro automático de auditorías.

9. RECOMENDACIONES

- Establecer una política estricta de ejecución y custodia para los respaldos lógicos generados mediante la utilidad `mysqldump`. Es imperativo que estos archivos `.sql` se almacenen en repositorios en la nube (fuera de las instalaciones físicas) como plan de recuperación ante desastres (Disaster Recovery Plan).
- Para garantizar el rendimiento óptimo del servidor MySQL ante el crecimiento proyectado de las operaciones de la empresa, se sugiere implementar un *Connection Pool* (Piscina de conexiones) en la capa de aplicación web para gestionar las solicitudes concurrentes de los cajeros y vendedores.
- Programar ventanas de mantenimiento preventivo trimestrales enfocadas en la tabla `auditoria`. Se recomienda que los registros históricos de acceso y modificaciones se migren periódicamente a un sistema de almacenamiento en frío (Data Warehouse) para evitar la degradación del rendimiento en las consultas operativas diarias.
- Se recomienda vincular el backend del nuevo sistema web a un servicio de notificaciones automatizadas que alerte a la gerencia cuando la vista `ProductosBajoStock` detecte artículos por debajo de su umbral mínimo, optimizando así los tiempos de reabastecimiento.

10. BIBLIOGRAFÍA

- Documentación oficial de MySQL: <https://dev.mysql.com/doc/>
- Elmasri, R., & Navathe, S. B. (2017). Fundamentos de sistemas de bases de datos. Pearson.
- Connolly, T., & Begg, C. (2015). Database Systems: A Practical Approach to Design, Implementation, and Management. Pearson.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts. McGraw-Hill.
- MySQL Tutorial: <https://www.mysqltutorial.org/>
- W3Schools MySQL: <https://www.w3schools.com/mysql/>

11. ANEXOS

Repositorio de GitHub :

<https://github.com/DanielV593/Proyecto-Base-de-Datos---Maria-Jose-Paredes---Juan-Vasquez>

